TSINGHUA SCIENCE AND TECHNOLOGY
ISSN 1007-0214 06/37 pp101–124
DOI: 10.26599/TST.2024.9010185
Volume 31, Number 1, February 2026

Differential Evolution with Joint Adaptation of Mutation Strategies and Control Parameters via Distributed Proximal Policy Optimization

Wenjie Ding, Mengtao Qian, Chao Lu, Jin Yi*, Huayan Pu, and Jun Luo

Abstract: The mutation operations and related control parameters play important roles in the performance of the differential evolution algorithm. Learning optimal policies for these strategies and parameters through reinforcement learning is a hot topic. However, most of the current studies focus on either mutation strategy selection or the control parameters alone while the others keep fixed or self-adaptive, resulting in deteriorated performances. To address this gap, this paper proposes a framework for the joint adaptation of mutation strategies and related control parameters based on deep reinforcement learning. In this method, the distributed proximal policy optimization algorithm is employed to train the agents to dynamically select the optimal combination of mutation strategies and control parameters. To enhance the agent's learning of the optimal policy, information derived from fitness landscape analysis is incorporated into the state representations. The training is conducted on the black-box optimization benchmark test problems, which are capable of generating large-scale test instances. Numerical results on the new problems from CEC2013 and CEC2017 test suites, and the real-world application of rover trajectory planning demonstrate that the proposed approach achieves competitive performance compared to state-of-the-art methods. The adaptation behavior and the contribution of learning are also thoroughly analyzed.

Key words: Differential Evolution (DE); Evolutionary Algorithm (EA); Deep Reinforcement Learning (DRL); parameter control

1 Introduction

With the advantages of easy deployment and a small number of hyperparameters, Differential Evolution (DE) stands out among many Evolutionary Algorithms (EAs)[1]. Various advanced DE-based algorithms have been widely applied in many real-world problems in engineering and scientific research[2, 3]. DE and its variants have achieved good rankings in recent evolutionary computation competitions[4]. Based on strategies to improve the performance of DE, its variants can be divided into three primary categories.

(1) Hybrid DE variants with other meta-heuristic algorithms. Mohamed et al.[5] proposed a hybrid DE variant named APGSK-IMODE and won the IEEE

• Wenjie Ding, Mengtao Qian, Huayan Pu, and Jun Luo are with State Key Labortary of Mechanical Transmission for Advanced Equipment and School of Mechanical and Vehicle Engineering, Chongqing University, Chongqing 400044, China. E-mail: 202307131143T@stu.cqu.edu.cn; 202107131361@stu.cqu.edu.cn; Phygood_2001@shu.edu.cn; luoj@cqu.edu.cn.
• Chao Lu is with School of Computer Science, China University of Geosciences, Wuhan 430078, China. E-mail: luchao@cug.edu.cn.
• Jin Yi is with State Key Laboratory of Mechanical Transmission for Advanced Equipment and School of Mechanical and Vehicle Engineering, Chongqing University, Chongqing 400044, China, and also with State Key Laboratory of Fluid Power and Mechatronic Systems, Zhejiang University, Hangzhou 310027, China. E-mail: yijin@cqu.edu.cn.
* To whom correspondence should be addressed.
Manuscript received: 2024-04-19; revised: 2024-09-26; accepted: 2024-09-29

© The author(s) 2026. The articles published in this open access journal are distributed under the terms of the Creative Commons Attribution 4.0 International License (http://creativecommons.org/licenses/by/4.0/).

CEC 2021 competition. This algorithm was designed to improve performance by integrating the Adaptive Parameters Gaining Sharing Knowledge (APGSK) with an Improved Multi-Operator Differential Evolution (IMODE). Deng et al.[6] proposed an improved differential evolution, namely HMCFQDE, combining the quantum computing characteristics of the Quantum Evolutionary Algorithm (QEA) and the divide-and-conquer idea of the Cooperative Coevolution Evolutionary Algorithm (CCEA). Hachana and El-Fergany[7] proposed an improved hybridized Artificial Bee Colony Differential Evolution optimizer (ABCDE). Ustun et al.[8] proposed a hybrid Artificial Bee Colony (ABC) algorithm with DE, the exploitation ability of ABC is enhanced by replacing the onlooker bee operator with those of mutation and the crossover phases of DE. Li et al.[9] proposed an efficient DE variant named OLELS-DE, by designing orthogonal learning and elites local search mechanisms to effectively relieve the stagnation and premature convergence problems in DE. In 2023, Yu et al.[10] proposed a Hybrid Grey Wolf Optimizer and Differential Evolution (HGWODE) algorithm to solve the UAV path planning problem. Huang et al.[11] proposed an innovative approach that integrates an adaptive cylinder vector Particle Swarm Optimization (PSO) algorithm with DE to tackle the problem of UAV path planning. Li et al.[12] proposed an enhanced DE algorithm that hybridizes with the Estimation-of-Distribution Algorithm (EDA). This approach aims to fully exploit the strong exploration capabilities of DE and the strong exploitation capabilities of the EDA, and a control parameter is adopted to control the number of offspring sampled by EDA to balance exploitation and exploration.

(2) DE variants with novel operation strategies. As we know, mutation strategy plays a crucial role in offspring generation and algorithm performance[13]. Various mutation operations have been proposed to improve the quality of offspring, such as "DE/best/1", "DE/best/2", and "DE/current-to-best/1"[14]. Each Mutation Strategy (MS) has its strengths, with "DE/rand/1" being more exploration-oriented and "DE/best/1" being more exploitation-oriented. However, no single MS is perfect, and new strategies have been proposed to overcome their shortcomings and enhance offspring quality[15−17]. The most representative mutation strategy "DE/current-to-pbest" with an external archive of inferior solutions is introduced into JADE[18]. This strategy, coupled with the most representative adaptive parameter control, continues to serve as the foundation for the development of new DE-based variants. Some other novel mutation strategies have been proposed in recent years. Stanovov et al.[19] proposed LSHADE-RSP, which utilizes a rank-based selective pressure approach in its mutation strategy, as opposed to the random selection method. In 2019, Mohamed et al.[20] introduced two novel mutation strategies to LSHADE and SHADE, namely the less greedy mutation strategy of DE/current-to-ord-best/1 (ord stands for ordered), and the more greedy mutation strategy of DE/current-to-ord-pbest/1. In 2020, Nadimi-Shahraki et al.[21] proposed an new DE variant named Multi-Trial vector-based Differential Evolution (MTDE) by introducing an adaptive movement step designed based on a new multi-trial vector approach, which combines different search strategies in the form of trial vector producers. Deng et al.[22] proposed an improved quantum-inspired differential evolution, namely MSIQDE algorithm, based on making use of the merits of the Mexh wavelet function, standard normal distribution, adaptive quantum state update, and quantum nongate mutation, to avoid premature convergence and improve the global search ability. Sallam et al.[23] proposed an IMODE algorithm, starting by dividing the initial population into several sub-populations, and each sub-population evolving its own DE variant. In 2021, Zeng et al.[24] proposed a new selection operator to improve the differential evolution algorithm's ability to escape the local optimal value. Kumar et al.[25] proposed a novel DE variant named OLSHADE-CS. This variant utilizes orthogonal arrays and a neighborhood search for the initial formation of its population. Further enhancements are achieved through a conservative selection scheme along with integration of multiple mutation strategies in the OLSHADE-CS. Duan et al.[26] proposed a DE algorithm with a Superior-Inferior mutation scheme (namely SIDE) to solve global optimization problems over continuous space.

(3) DE variants with adaptive parameters and operations. The control parameter F, Crossover Rate (CR), as well as the mutation strategy selection significantly affect DE's performance. In the early research on DE, the parameter adjustment was typically performed using a trial-and-error approach, where the control parameters were tested on a set of optimization problems to determine the optimal values.

This method, often referred to as "parameter tuning", involved continuous testing, and once the parameters were determined, they remained fixed throughout the optimization process[27]. Further research has revealed that achieving the optimal configuration of control parameters in DE requires considering not only the characteristics of different problems but also the different states of the evolutionary search process[17, 28, 29]. Consequently, the domain of DE research has expanded to include the development of "parameter control" methods, which enable the dynamic updating of control parameters during the evolution[30]. Scholars have explored various parameter adaptation mechanisms, including those based on normal distribution[31], Cauchy distribution[18], uniform distribution[32, 33], and other variants[34–36]. As stated by Karafotias et al.[27], "parameter tuning" and "parameter control" are two sides of the same coin, with the former being stationary and the latter being non-stationary. It can be argued that "parameter control" offers more advantages compared to "parameter tuning", as it allows for the collection of fitness landscape information in the early stages of evolution and actively guides the algorithm towards better performance in later stages. However, the problem of parameter control remains a challenging task particularly for general parameter control methods[37]. Cao et al.[38] proposed an adaptive differential evolution algorithm, which harnesses the standard deviation of the fitness values and the aggregate standard deviation across each population dimension to deeply mine and effectively use the demographic trait data. Zhang et al.[39] proposed a unified mutation operator with learnable parameters for DE. The proposed operator is adaptable through parameter settings and can evolve into various classic mutation operators, thus achieving simultaneous parameter control and operator selection. Meng[40] introduced a novel DE algorithm with dimension improvements based adaptation schemes for control parameter F and CR. Li et al.[4] introduced a Population State Evaluation (PSE)-based improvement framework for DE algorithms to address the issues of premature convergence and stagnation. Zhang et al.[41] proposed the multiple strategy improved DE algorithm for addressing the challenge of UAV trajectory planning in complex terrains. This algorithm incorporates three strategies and dynamically adjusts the values of F and CR.

Recent studies have highlighted the potential of Reinforcement Learning (RL) in addressing the challenges of parameters adaptation[42, 43]. In general, RL can be effectively integrated into the EA framework for its strong learning capability and low time complexity during the testing stage[44]. Lacerda et al.[45] conducted a systematic survey of general parameter control methods for EAs and discovered that nearly two-thirds of these methods were based on RL. Sharma et al.[46] applied a Double Deep Q-Networks (DDQN) to dynamically select mutation strategies in DE, presenting valuable design insights into reinforcement learning techniques. Tian et al.[47] introduced a deep RL-based adaptive operator selection strategy for the decomposition-based Multi-Objective Evolutionary Algorithm (MOEA). In their work, a Deep Q-Network (DQN) instead of the Q-table is used to learn a policy for estimating the Q values of a given state-action pair. Tan and Li[48] proposed an improved DE algorithm with mixed mutation strategy based on DQN (DEDQN), in which a DQN algorithm realizes the adaptive selection of mutation strategy in the evolution process. Peng et al.[49] proposed an adaptive differential evolution algorithm that integrates Q-learning to learn the optimal mutation strategy selection strategy. Some other works focus on obtaining the optimal parameter control strategies. Sun et al.[50] proposed a learning method that learns a parameter control strategy from previous optimization experience, which uses the policy gradient to train it. Liu et al.[37] proposed an LADE algorithm, leveraging policy gradient methods to learn from optimization experiences, thereby dynamically deciding control parameters that adapt to changing problem landscapes.

The combination of RL and DE has also been used to tackle real-world problems, such as renewable energy systems, scheduling, and structural optimization[51–54]. Hu et al.[55] used Q-learning to control the scaling factor F of DE which shows good performance in the PhotoVoltaic (PV) model. Xu et al.[51] proposed an RL-based DE for the energy scheduling problem of an integrated energy system. In this method, the mutation operators and their associated parameters are dynamically selected through the Q-learning algorithm. Huynh et al.[53] proposed a parameter adaptive strategy for adjusting F and CR in DE based on Q-learning and applied it to structural optimization problems. Li et al.[56] proposed an RL-based multi-objective DE to address the scheduling problem of continuous

annealing. Mughees et al.[57] proposed a smart Integrated Demand Response (IDR) program for industrial microgrids. RL has also been combined with other EAs/meta-heuristics, including the grey wolf optimizer[58], the estimation of distribution algorithm[59], multi-objective particle swarm optimization[60], and the artificial bee colony algorithm[61].

From the aforementioned studies, it can be observed that RL is predominantly used to learn an optimal policy either on mutation strategy selection or parameter control (i.e., F and CR) in DE. This is likely due to the increased complexity of the learning task when dealing with a larger action space. However, the performance of DE is influenced by both mutation strategy selection and control parameters. Therefore, limiting adaptation to only one of these aspects may restrict overall performance[27]. Consequently, there is a need for an optimal joint adaptation policy that simultaneously addresses mutation strategy selection and control parameters.

To fill the gap of learning the optimaljoint mutation and parameter policy via RL, we propose a unified adaptation framework in this study, named DRLDE, which uses Deep Reinforcement Learning (DRL) to address both mutation strategies and control parameters. The DRLDE approach employs the distributed Proximal Policy Optimization (PPO) algorithm to train agents, enabling the adaptive selection of synthesized configurations for both mutation strategies and control parameters. To better learn the optimal policy, the state representations incorporate information obtained from fitness landscape analysis. The training process is conducted on the Black-Box Optimization Benchmark (BBOB) test problems, selected for their ability to generate large-scale test instances. A single offline training session on the designated training function is sufficient to acquire the trained model, facilitating online application. Empirical validation of DRLDE is performed through extensive numerical experiments, including the CEC2013 and CEC2017 test suites, as well as a sophisticated 60-dimensional real-world problem focused on rover trajectory planning. The experimental results demonstrate that DRLDE is competitive compared to several other prominent DE variants and advanced heuristic algorithms.

The major contributions of this study are summarized as follows:

(1) To the best of the authors' knowledge, this study represents the first attempt to utilize reinforcement learning for simultaneous control over the mutation strategy, F, and CR within the DE algorithm.

(2) We propose a novel DE variant named DRLDE, which uses the distributed PPO algorithm to enable adaptive selection of synthesized configurations for both mutation strategies and control parameters. The state representations in DRLDE integrate fitness landscape analysis to measure problem characteristics, and utilize optimization process statistics and population diversity to assess the optimization status. This combination provides a comprehensive understanding of the problem landscape and the current optimization process, guiding the evolution effectively.

(3) The DRLDE algorithm is trained using 750 instances from the BBOB test suite. Extensive numerical experiments on the CEC2013 and CEC2017 test suites, and a sophisticated 60-dimensional real-world problem (rover trajectory planning) demonstrate that DRLDE is competitive compared to several other prominent DE variants and recent advanced heuristic algorithms.

The rest of this paper is structured as follows. Section 2 provides the background of DE, deep reinforcement learning, and fitness landscape analysis. Section 3 outlines the specifics of the proposed method, encompassing the modeling process, distributed PPO, and working steps. In Section 4, we detail the experimental setup and results, including comparisons with state-of-the-art methods, an analysis of the adaptation behavior, an evaluation of the learning contributions, and the computational complexity analysis. Section 5 demonstrates the application of the trained DE solver for the rover trajectory planning problem. Section 6 discusses the distinctions between our proposed approach and previous works. Finally, Section 7 concludes the paper and discusses future research directions.

2 Preliminary

This study aims to develop a DRL-enhanced DE optimizer, in which the DRL agents interact with the environment and monitor state information, including the fitness landscape, to learn an optimal joint adaptation policy for mutation strategies and parameters F and CR. This section will first provide an overview of the preliminaries, including the DE algorithm, DRL, and fitness landscape analysis, to

better contextualize this study.

2.1 DE algorithm and its variants

In general, we are aiming to solve the following global black-box minimization problems $f(\vec{x})$:

$$\begin{aligned} &\text{minimize: } f(\vec{x}),\\ &\text{subject to: } L_i \leqslant x_i \leqslant U_i,\ 1 \leqslant i \leqslant D \end{aligned} \tag{1}$$

where $\vec{x}=(x_1, x_2, \ldots, x_D)$ is the decision vector, x_i $(1 \leqslant i \leqslant D)$ is the decision variable for the i-th dimension, and D is the dimension of variables. L_i and U_i are the lower and upper bounds of the decision variables. The above-mentioned problems exhibit several characteristics. Firstly, no derivative information is available for $f(\vec{x})$, therefore, previous gradient-based optimization algorithms can not be directly utilized. Secondly, the fitness landscape of $f(\vec{x})$ can vary from problem to problem, necessitating the optimizer to be effective, efficient, and robust.

DE, proposed by Storn and Price[62], is a population-based evolutionary algorithm that is well-suited for solving the aforementioned problems. A DE population consists of N individuals $\vec{x}_{i,g}$ $(i=1, 2, \ldots, N)$, where g represents the generation number. Each individual $\vec{x}_{i,g}=(x_1, x_2, \ldots, x_D)$ represents a solution to the problem $f(\vec{x})$. The DE population evolves through three main steps: mutation, crossover, and selection.

2.1.1 Mutation

The mutation operator of the original DE is "DE/rand/1", and then many novel mutation strategies appear in DE variants, such as "DE/best/1", "DE/rand/2", "DE/best/2", and "DE/current-to-best/1".

Suppose $\boldsymbol{P}$ is a DE population, and $\vec{x}_{\text{best}}$ denotes the individual with the best fitness among $\boldsymbol{P}$. Let $\vec{x}=(x^1, x^2, \ldots, x^N)\in\mathbf{R}^N$ be an individual solution in $\boldsymbol{P}$. To obtain an offspring individual $\vec{u}=(u^1, u^2, \ldots, u^N)\in\mathbf{R}^N$, first, different forms of mutation strategies are executed to obtain a donor vector $\vec{v}_i$,

(1) "DE/rand/1":

$$\vec{v}_{i,g}=\vec{x}_{r_1,g}+F\cdot(\vec{x}_{r_2,g}-\vec{x}_{r_3,g}) \tag{2}$$

(2) "DE/best/1":

$$\vec{v}_{i,g}=\vec{x}_{\text{best},g}+F\cdot(\vec{x}_{r_1,g}-\vec{x}_{r_2,g}) \tag{3}$$

(3) "DE/rand/2":

$$\vec{v}_{i,g}=\vec{x}_{r_1,g}+F\cdot(\vec{x}_{r_2,g}-\vec{x}_{r_3,g})+F\cdot(\vec{x}_{r_4,g}-\vec{x}_{r_5,g}) \tag{4}$$

(4) "DE/best/2":

$$\vec{v}_{i,g}=\vec{x}_{\text{best},g}+F\cdot(\vec{x}_{r_1,g}-\vec{x}_{r_2,g})+F\cdot(\vec{x}_{r_3,g}-\vec{x}_{r_4,g}) \tag{5}$$

(5) "DE/current-to-best/1":

$$\vec{v}_{i,g}=\vec{x}_{i,g}+F\cdot(\vec{x}_{\text{best},g}-\vec{x}_{i,g})+F\cdot(\vec{x}_{r_1,g}-\vec{x}_{r_2,g}) \tag{6}$$

(6) "DE/current-to-pbest/1":

$$\vec{v}_{i,g}=\vec{x}_{i,g}+F\cdot(\vec{x}^{p}_{\text{best},g}-\vec{x}_{i,g})+F\cdot(\vec{x}_{r_1,g}-\tilde{x}_{r_2,g}) \tag{7}$$

(7) "DE/current-to-rand/1":

$$\vec{u}_{i,g}=\vec{x}_{i,g}+K\cdot(\vec{x}_{r1,g}-\vec{x}_{i,g})+F\cdot(\vec{x}_{r_2,g}-\vec{x}_{r_3,g}) \tag{8}$$

(8) "DE/rand-to-best/2":

$$\begin{aligned}\vec{v}_{i,g}=\vec{x}_{r1,g}+F\cdot(&\vec{x}_{\text{best},g}-\vec{x}_{r1,g}+\vec{x}_{r2,g}-\vec{x}_{r3,g}+\\ &\vec{x}_{r4,g}-\vec{x}_{r5,g})\end{aligned} \tag{9}$$

where F $(0<F<1)$ is the scaling factor, $\vec{x}_{r_1}\sim\vec{x}_{r_5}$ are randomly selected distinct individuals from $\boldsymbol{P}$, $\vec{x}^{p}_{\text{best},g}$ is a randomly selected elite individual from the top Np ones in the g-th generation, and $\tilde{x}_{r_2,g}$ represents a randomly chosen individual from the combined set of the current population and its external archive, K is a randomly generated number within the interval $[0,1]$.

2.1.2 Crossover

Subsequently, the mutant individuals will generate offspring solution $\vec{u}$ through crossover operations, with a crossover rate of CR between 0–1[63]. Generally, DE employs exponential and binomial crossover operations. Due to its robustness and effectiveness, the binomial crossover scheme has become commonly used in recent studies, as shown in the following formula[64]:

$$\vec{u}_i^{\,g}=\begin{cases}\vec{v}_i^{\,g}, & \text{if rand}(\,)\leqslant \text{CR or } j=j_{\text{rand}};\\ \vec{x}_i^{\,g}, & \text{otherwise}\end{cases} \tag{10}$$

where $\text{rand}(\cdot)$ is a random number in range of $[0,1]$, j_{rand} is a random integer within $[1,D]$.

2.1.3 Selection

In the selection operation of DE, the "survival of the fittest" criterion is adopted. In other words, in the next generation, solutions with better fitness will be retained, while solutions with poorer fitness will be discarded. For a problem aiming to minimize, the selection operation is illustrated as follows:

$$\vec{x}_i^{\,g+1}=\begin{cases}\vec{u}_i^{\,g}, & \text{if } f(\vec{u}_i^{\,g})\leqslant f(\vec{x}_g^{\,g});\\ \vec{x}_i^{\,g}, & \text{otherwise}\end{cases} \tag{11}$$

2.1.4 Termination

When the stopping criterion is met, the iteration process of DE will be terminated and the optimal solution will be output. Common termination criteria are like the maximum number of function evaluations and iterations, abbreviated as MAXNFE and MAXNI.

2.2 RL and Deep RL

RL is a machine learning paradigm that focuses on learning and decision-making in a Markov Decision Process (MDP). In RL, there are four fundamental elements: state s, action a, reward r, and transition probability p, collectively forming a 4-tuple (s, a, r, p). The policy π represents a conditional probability distribution denoted as $\pi = p(a_t|s_t; \theta)$ (θ defined as the hyper-parameter of π). Given the current state s_t at time step t, the agent selects an action a_t by randomly sampling from π. After taking this action, the agent observes an updated state s_{t+1} and receives an immediate reward r_{t+1}. The goal of an RL task is to develop an optimal policy π^* that maximizes the return, which is defined as the total discounted future reward,

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \tag{12}$$

where G_t denotes the return and γ denotes the weight factor.

With the powerful capability of representation, deep neural networks are potential alternatives for function approximation. When utilizing deep neural networks to construct the policy function in RL, it becomes the so-called deep RL. Many deep RL algorithms have been developed in recent years, popular ones among them include Deep Deterministic Policy Gradient[65], Proximal Policy Optimization[66], Soft Actor-Critic[67], and Asynchronous Advantage Actor-Critic (A3C).

2.3 Fitness landscape analysis

The Fitness Landscape (FL) theory proposed by Wright[68] has created a new analytical method for optimization problems. According to Tan and Li[48], FL analysis has the same efficacy in EAs. Generally, an FL is a static representation of a problem, consisting of the fitness function values of all individuals scattered throughout the decision space. The FL could be different for different types of problems or at different stages of evolutionary search[69]. Therefore, the study of the FL has a guiding role in the field of evolutionary computing. Many metrics based on FL analysis have been applied to estimate the characteristics of optimization problems, including information entropy measure, Fitness Distance Correlation (FDC), and fitness cloud[70].

FDC is an FL analysis method that employs fitness values and the distance relationship between the solution and the optimal solution to measure the complexity of the problem[71]. Therefore, FDC is adopted in this paper. The metric is the correlation coefficient between the distance from the solution in the exploration space to the nearest global optimal solution and the fitness value. Suppose there is a sample of m solution-fitness pairs, forming a sequence $F = \{f_1, f_2, \ldots, f_m\}$ and corresponding fitness values. The Euclidean distance $D = \{d_1, d_2, \ldots, d_m\}$ is calculated by the distance from the optimal individual to all other ones in the sample. Hence, FDC can be calculated by the following formula[72]:

$$\mathrm{FDC} = \frac{C_{F,D}}{\delta_F \cdot \delta_D} \tag{13}$$

where δ_F and δ_D represent the variances of F and D, respectively. $C_{F,D}$ denotes the covariance of F and D, which is defined as

$$C_{F,D} = \frac{1}{m} \sum_{i=1}^{m} (f_i - \overline{f})(d_i - \overline{d}) \tag{14}$$

where $\overline{f}$ and $\overline{d}$ are the mean values of F and D, respectively.

Calculating the global FL of an optimization problem is a difficult task, and a random walk sampling algorithm is usually adopted[48].

3 Proposed Approach

In this section, a new DE variant is developed, featuring a joint adaptation of mutation strategy and parameters. The optimization process is initially modeled as an MDP, with specific RL settings including state, action, and reward function. Subsequently, the joint adaptation strategy undergoes training using distributed PPO. Ultimately, the trained model is integrated into the DE framework.

3.1 Model the evolutionary search process as MDP

To learn the optimal policy for the agent, the parameters of the neural network are obtained through training. Consequently, it becomes essential to model the DE algorithm's evolutionary search process as an MDP, encompassing environment, state, action, policy, and rewards[50].

3.1.1 Environment

Throughout the training phase, the DE agent is anticipated to accumulate experiences by resolving a variety of optimization problems, thereby constituting

the environment as a compilation of diverse optimization challenges, known as training functions. These training functions are utilized to assess the controller's performance during the learning process. Figure 1 shows the whole procedure the DE agent interacts with the environment.

3.1.2 State

A state value should furnish the agent with pertinent and ample information regarding the environment, enabling the selection of an optimal action based on the current state. In the proposed environment, we utilize a combination of eigenvalues, which depict the optimization solution process, and a fitness landscape, which illustrates the optimization problems' difficulty, to define our state values. In addition, the diversity of the population at a certain moment can reflect the state of the population at that moment to some extent, which is important to the solution of the dynamic optimization problem. Therefore, we add the state value of population diversity. Finally, a state tuple consisting of 8 elements is defined as follows:

- $S_t[1] = \dfrac{\overline{f} - f_{\text{bsf}}}{f_{\text{wsf}} - f_{\text{bsf}}}$, where $\overline{f}$ is the average fitness of the current population, f_{bsf} and f_{wsf} denote the best and worst fitness found up to this step, respectively.
- $S_t[2] = \dfrac{f_{\text{median}} - f_{\text{bsf}}}{f_{\text{wsf}} - f_{\text{bsf}}}$, where f_{median} is the median fitness of the current population.
- $S_t[3] = \dfrac{\text{std}_{\text{F}}}{\text{std}_{\text{max}}}$, where std_{F} denotes the standard deviation of the fitness within the whole population, std_{max} represents the maximum of std_{F}, calculated by the worst and the best fitness so far.
- $S_t[4] = \dfrac{\text{budget}}{\text{budget}_{\text{max}}}$, where $\text{budget}_{\text{max}}$ is the maximum function evaluations per run, budget is the current function evaluations.
- $S_t[5] = \dfrac{\text{dim}}{\text{dim}_{\text{max}}}$, where dim denotes the problem scale, dim_{max} is the maximum dimension among all the benchmark functions.
- $S_t[6] = \dfrac{\text{stag}}{\text{stag}_{\text{max}}}$, where stag_{max} denotes the maximum number of generations, and the stag represents the number of generations that the best fitness has not been improvement.
- $S_t[7] = \dfrac{\text{diversity}}{\text{diversity}_{\text{ini}}}$, where diversity denotes the diversity of current population, $\text{diversity}_{\text{ini}}$ denotes the initial population diversity during evolution.
- $S_t[8] = \text{FDC}$, where FDC is the fitness distance correlation.

3.1.3 Action (A_t) and policy (π)

In an MDP, given the state S_t, the agent is capable of selecting an action from the policy π. For this study, the action A_t is defined as a combination of mutation strategy and control parameters (F and CR).

(1) MS

Various MSs have their unique benefits in DE. "DE/rand/1" is the most widely used MS, which is strong in exploration. According to Ref. [18], "DE/current-to-pbest/1" has a good performance in solving multi-modal problems when the global basin is initially discovered or in uni-modal problems. In

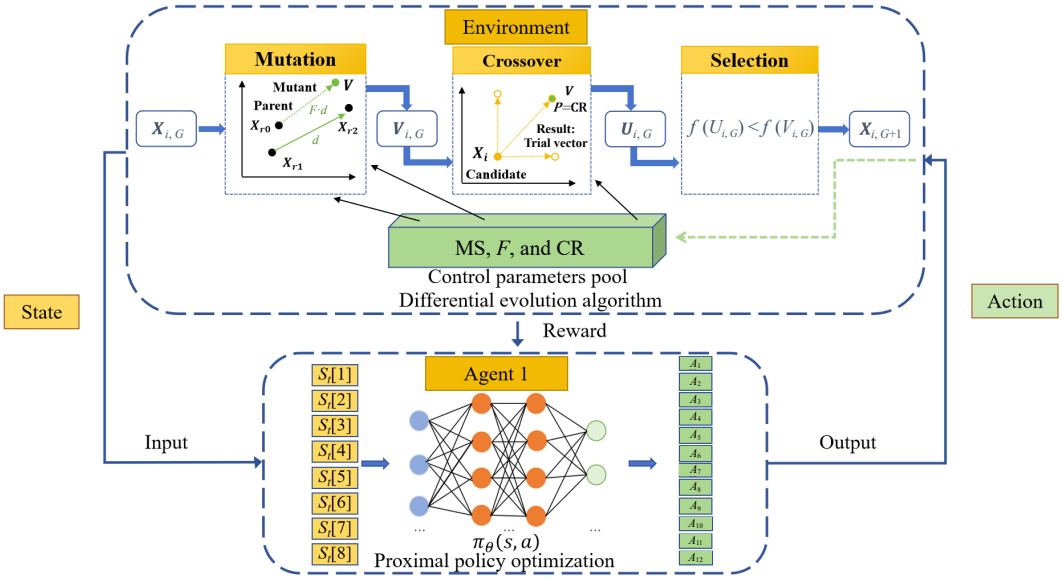


Fig. 1 Architecture of DRLDE.

contrast, "DE/current-to-rand/1" is an effective MS for solving rotated problems because it is rotation-invariant[73]. "DE/rand-to-best/2" is an MS that uses random individuals as the basis vector to converge to the optimal individuals. The difference transformation contains five random vectors, which balances the convergence speed and exploration well[46]. Therefore, the proposed algorithm adopts the above four MSs, namely "DE/rand/1", "DE/current-to-pbest/1", "DE/current-to-rand/1", and "DE/rand-to-best/2".

(2) Control parameters *F* and CR

Many previous researches have studied the impacts of F and CR. With the increase of F, the greater the difference between individuals, the more thorough the exploration of search space by the population, and the population diversity will increase. Therefore, a larger value of F can help the algorithm escape local optimization[74]. The crossover rate CR determines the mixing probability of the trial vectors and the target vectors. Higher CR can speed up convergence[75]. For separable problems, CR is best in the range of $(0, 0.2)$. On the contrary, CR is best in the range of $(0.9, 1.0)$ for multi-modal, parameter-dependant problems[63, 74]. For the proposed approach, we choose three parameter settings: $[F = 1.0,\ \mathrm{CR} = 0.1]$ is used to solve separable problems, $[F = 0.8,\ \mathrm{CR} = 0.2]$ can better balance the speed of population convergence and the degree of exploration of search space, $[F = 1.0,\ \mathrm{CR} = 0.9]$ is mainly used to preserve population diversity, enhance exploration ability, and avoid falling into local optimization[76].

Therefore, there are four MSs and three control parameter settings. The action is a combination of the above MSs and parameter settings, forming a twelve-dimensional discrete action space (see Table 1). Each action corresponds to a mutation strategy and parameter setting, so each action has different advantages. Through this approach, we achieved simultaneous control of three parameters without increasing the dimension of the action space (i.e., using only one discrete action space). According to different problems and differential stages of evolution, the agent can select appropriate actions from the action space, to get a better solution.

For policy, π, a fully connected neural network with 8-dimensional inputs and 12-dimensional outputs is constructed as $\pi(A_t|S_t;\ \theta)$.

3.1.4 Reward (r_{t+1})

In RL, the reward function's configuration is pivotal, exerting a direct impact on an algorithm's convergence behavior. The reward function evaluates the decision's quality when an agent acts upon the information available in a state. A higher reward value signifies a more advantageous decision by the agent in that specific state. Essentially, the reward function steers the learning direction of agents. In this study, the reward r_{t+1} is defined as follows:

$$r_{t+1} = \frac{\text{best}\{F_t\} - \text{best}\{F_{t+1}\}}{\text{best}\{F_t\}} \tag{15}$$

where best $\{F_t\}$ represents the current best fitness achieved at generation t, while best $\{F_{t+1}\}$ represents the best fitness obtained at generation t+1. From Eq. (15), it is evident that the magnitude of fitness enhancement is directly proportional to the molecular component. A diminished fitness value correlates to a reduced denominator, amplifying the influence of the numerator. This implies that throughout the evolutionary procedure, the reward weight exhibits an increasing trend. As corroborated by various studies[27, 77, 78], as the evolutionary process progresses, enhancing fitness becomes increasingly challenging. This design architecture, therefore, aids in circumventing premature convergence to local optima, thus enhancing global optimization capacities. Consequently, the reward function encapsulates the relative fitness augmentation from generation t to generation $t+1$ post adaptive parameter modulation. An elevated reward metric signifies the algorithm's enhanced performance as a result of adaptive parameter control.

Table 1 12-dimensional discrete actions for DRLDE.

Index	Action	Index	Action
0	"DE/rand/1", F = 1.0, CR = 0.1	6	"DE/current-to-rand/1", F = 1.0, CR = 0.1
1	"DE/rand/1", F = 1.0, CR = 0.9	7	"DE/current-to-rand/1", F = 1.0, CR = 0.9
2	"DE/rand/1", F = 0.8, CR = 0.2	8	"DE/current-to-rand/1", F = 0.8, CR = 0.2
3	"DE/current-to-pbest/1", F = 1.0, CR = 0.1	9	"DE/rand-to-best/2", F = 1.0, CR = 0.1
4	"DE/current-to-pbest/1", F = 1.0, CR = 0.9	10	"DE/rand-to-best/2", F = 1.0, CR = 0.9
5	"DE/current-to-pbest/1", F = 0.8, CR = 0.2	11	"DE/rand-to-best/2", F = 0.8, CR = 0.2

3.2 Learning the adaptation strategy via distributed PPO

PPO[66] is a policy gradient method for RL that continuously interacts with the environment to sample data and employs stochastic gradient ascent to optimize a "surrogate" objective function. This study utilizes the PPO-clip algorithm for policy optimization.

PPO-clip typically involves multiple SGD steps to maximize the objective as expressed in the following:

$$\theta_{k+1} = \arg\max_{\theta} \underset{s,\, a\sim\pi_{\theta_k}}{E} [L(s,\, a,\, \theta_k,\, \theta)] \tag{16}$$

where π_{θ_k} denotes the policy network with parameter θ_k, and $(s,\, a)$ represents the state-action pair sampled from π_{θ_k}. E signifies the empirical average over a finite batch of samples. The surrogate objective L is calculated as

$$L(s,\, a,\, \theta_k,\, \theta) = \min\left(\frac{\pi_\theta\,(a\mid s)}{\pi_{\theta_k}\,(a\mid s)} A^{\pi_{\theta_k}}(s,\, a),\; g\,(\epsilon,\, A^{\pi_{\theta_k}}(s,\, a))\right) \tag{17}$$

where

$$g\,(\epsilon,\, A) = \begin{cases} (1+\epsilon)A, & A \geqslant 0; \\ (1-\epsilon)A, & A < 0 \end{cases} \tag{18}$$

where ϵ is a minor hyperparameter that permits a disparity between the new and old policy. The term $\dfrac{\pi_\theta\,(a\mid s)}{\pi_{\theta_k}\,(a\mid s)}$ indicates the ratio of probability distributions between the new and old policies. A is an advantage function used to evaluate the advantage of selecting action a over other actions in state s.

The primary steps of PPO are as follows: first, a set of trajectories are collected by executing policy π_{θ_k} in the environment; second, the rewards are computed according to these trajectories; third, advantage estimates A are calculated using the value network; next, the policy is updated by maximizing the objective in Eq. (16) through SGD; and finally, the process is repeated. For more details, readers can refer to Ref. [66].

PPO continuously optimizes the policy $\pi_\theta\,(a\mid s)$ by interacting with the environment. To a certain extent, it can be said that if more interaction data (experience) can be collected, then the learning of the policy can be accelerated. By interacting with the environment with multiple agents in parallel, we can gain multiple experiences in the same amount of time. Therefore, a distributed version of the PPO algorithm is used to accelerate the learning process as shown in Fig. 2.

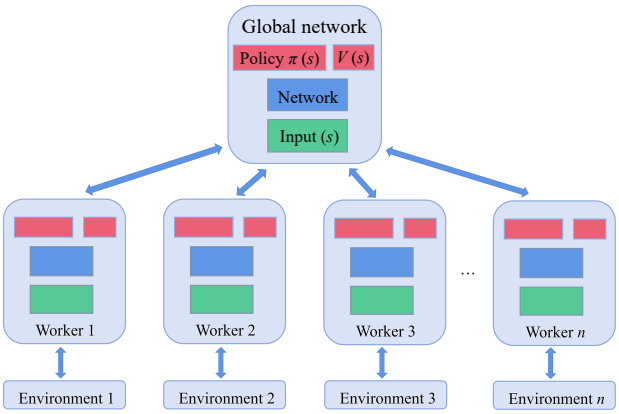


Fig. 2 Distributed training architecture.

In distributed PPO, multiple parallel workers (or agents) interact with independent instances of the environment to collect experience. This experience is then aggregated to update a central policy network. Algorithm 1 shows the offline training procedure. By employing multiple agents in parallel, the distributed version of PPO enables faster and more efficient policy optimization, leveraging the computational power of multiple workers to accelerate the learning process.

3.3 Embedding the optimized policy into DE framework

Upon completing the training phase, it is assumed essential knowledge for dynamically selecting mutation operations and control parameters, derived from optimization experiences, has been acquired. With a new test problem, the DE can be directly employed using this trained controller. This refined approach, termed DRLDE, is illustrated in Fig. 3. For a detailed depiction of the process, Algorithm 1 outlines all the working steps. Within Algorithm 2, the evolutionary process mirrors that of a conventional DE, but with a distinction: the mutation strategy and control parameters (F, CR) are derived as samples from the controller's output, as depicted in Lines 6 and 7.

4 Experimental Study

In this section, we first introduce the implementation details of the proposed method during the training and testing processes. Subsequently, we present the numerical experiment results in comparison with other advanced algorithms. Finally, we analyze the adaptive behavior of the proposed method, its learning contributions, and the computational complexity in detail.

Algorithm 1 Offline training procedure with distributed PPO

Input: neural network parameter θ, number of training episodes Q, training function set $\mathcal{F}$, population size N_p, number of workers N_w, maximum function evaluations MAXNFE, and update frequency freq
Initialize global policy network π_θ and value network V_ϕ uniformly at random;
episode = 0;
while episode < Q **do**
  for each worker **do** ▷ Collect data by distributed workers
    Recieve the current global policy network π_θ;
    Select a training function randomly from $\mathcal{F}$;
    Initialize population, set nfes = N_p;
    while nfes < MAXNFE **do**
      Get the action according to the local copy of the policy network π_{θ_k};
      Complete the mutation, crossover, and selection operator of the whole population with F, CR, and mutation strategy;
      Calculate reward according to Eq. (15);
      Collect trajectories $\{(s_t, a_t, r_t, s_{t+1})\}$;
      nfes = nfes + N;
    end while
  end for
  episode = episode + N_w;
  if episode % freq == 0 **then** ▷ Update global policy network on master program
    Aggregate trajectories from all workers;
    Compute rewards-to-go R_t and advantage estimates A_t using the global value network V_ϕ;
    Update global policy parameters θ by maximizing the PPO-clip objective using SGD;
    Update global value parameters ϕ by minimizing the value function loss;
    Synchronize the updated weights of the global policy and value networks across all workers;
  end if
end while
Output: Final global policy network parameters θ

4.1 Training and test settings

(1) Environment creation

We have developed a custom environment for our reinforcement learning tasks, seamlessly integrated within the OpenAI Gym framework[79]. The action spaces, environment states, rewards are defined according to descriptions in Section 3.1. One challenge with current RL algorithms, including PPO, is their requirement for extensive data interacting with the environment. Thus, identifying a test suite with diverse characteristics is crucial. In this study, the training functions from the BBOB test suite[80] are used. The BBOB benchmark comprises five distinct problem characteristics: separable functions, functions with different conditional actions, unimodal functions, multimodal functions, and functions with sufficient and insufficient global structures. Furthermore, the same function in BBOB can generate numerous different instances, thereby providing a wide array of training functions. Consequently, the training set encompasses a vast range of functions with diverse characteristics, which aids in developing algorithms with strong generalization capabilities. The training data consists of a total of 25 training functions (10-D, 30-D) and each function has 15 different instances, resulting in a total of 750 instances. At the beginning of each training episode, randomly select an instance from the training data for optimization. During the training process, the population size is set to $N = 50$ for 10-D problems and $N = 100$ for 30-D.

(2) Implementation of the distributed PPO

We implemented a distributed PPO using Stable Baseline3[81], an advanced platform for reinforcement learning. Our implementation employs the PPO-Clip algorithm, which is known for its stability in policy updates by utilizing a clipped objective function. With 50 parallel agents, each with its own environment instance, our setup significantly accelerates the data collection and learning process. The agents operate independently, computing gradients that are communicated to a central parameter server, which updates the global policy network. The neural network architectures for the policy and value networks are shown in Fig. 4. We finetuned the hyperparameters using the grid search, including the learning rate, batch size, discount factor, and clipping parameters, and the final settings are shown in Table 2.

(3) Test settings

After training, the best PPO agent is saved, including the parameters of the deep neural network, and then directly used for online testing. The test experiment consists of two parts: DRLDE is first compared with other DE variants on CEC2013 and CEC2017. Then, a comparative experiment is conducted between DRLDE and other advanced meta-heuristic algorithms. All calculation criteria refer to those described in Ref. [82]. Each algorithm is repeated 51 times on each function. The algorithm will terminate either the MAXNFE

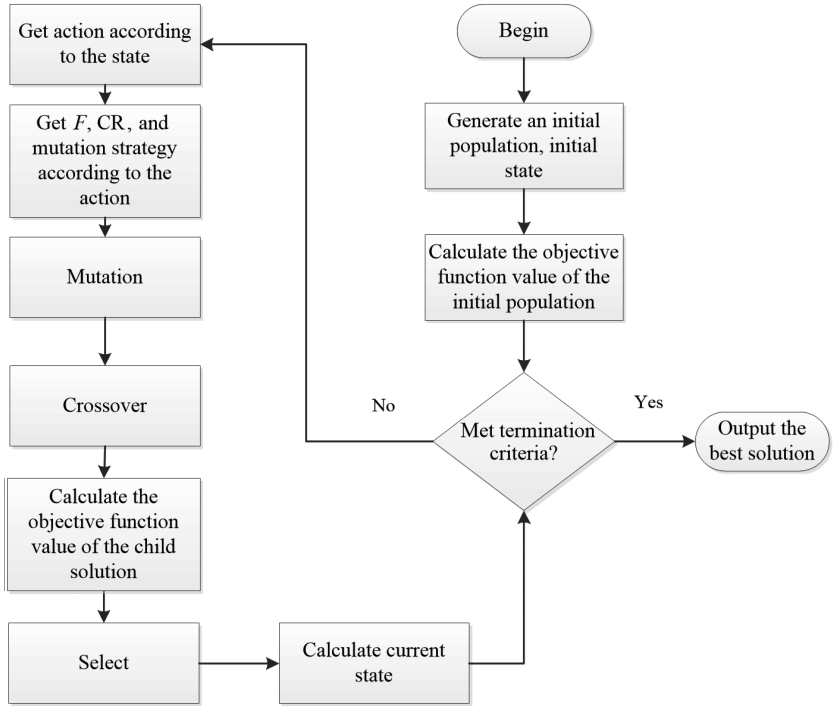


Fig. 3 Flowchart of DRLDE.

Algorithm 2 Pseudo code of online testing

Input: trained neural network parameter θ
Initialize population;
Evaluate the initial population;
Calculate the initial state;
while the termination criteria have been not met **do**
  Get the action according to the state by the trained neural network;
  Calculate F, CR, and mutation strategy according to the action (Described in Section 3.1);
  Generate the donor vector $\vec{V}$ using above mutation strategy and F ;
  Generate the child solution $\vec{U}$ with CR using Eq. (10);
  Select operator according to Eq. (11);
  Update state for next generation;
end while
Output: The best solution

exceeds $D\times10^4$ or the fitness error to the global optimum is smaller than 10^{-8}.

All the experiments are conducted on a high-performance computing platform with Intel Xeon Platinum 8180 CPU @ 2.50 GHz, 56 cores, and Python 3.

4.2 Advanced algorithms for comparison

Several well-known DE variants are selected in the

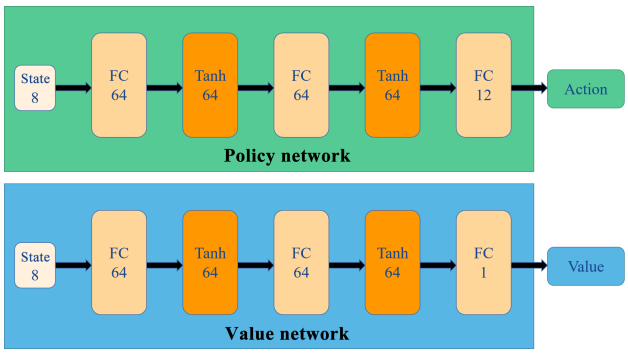


Fig. 4 Architectures for the policy and value networks. "State 8" indicates the number of states is 8; "FC 64" stands Fully Connected (FC) layer with 64 neurons; "Tanh 64" stands for the activation function of neural networks is hyperbolic Tangent function (Tanh), which outputs 64 layers; "FC 12" stands for FC layer with 12 neurons; and FC 1 is the output layer).

Table 2 Detailed parameter settings for the training.

Parameter	Value
Learning rate (α)	0.0003
Minibatch size	64
Discount (γ)	0.99
Generalized advantage estimator parameter (λ)	0.95
Number of epochs	10
Horizon (T)	2048
Clipping parameter (ϵ)	0.2

comparative experiment,which are shown as follows:

- **DE[62]:** The original DE proposed by Storn and Price and its mutation strategy is "DE/rand/1".
- **DE-random:** This algorithm adopts the same setting as ours except that it uses a random policy when selecting actions.
- **JADE[18]:** A famous adaptive DE algorithm in which the "DE/current-to-pbest/1" mutation is first proposed. An external archive is adopted to better guide the offspring to mutate. F is sampled from the Cauchy distribution based on historical successful experience (normal distribution for CR) and the adaptation of F and CR is applied to the individual level.
- **CoDE[76]:** CoDE defines three parameter settings and three mutation strategies, and randomly selects one parameter setting for each MS during evolution.
- **SHADE[83]:** An improved version of JADE. In this method, different parameter adaptive mechanisms based on success history are used to achieve adaptive control of F and CR. A historical memory with a capacity of H has been proposed, improving the robustness of the algorithm.
- **jDE[84]:** In the jDE algorithm, F and CR are controlled and changed with a certain probability to transmit better parameter values to future generations.
- **SaDE[85]:** In the SaDE algorithm, the adaptive mechanism of the two mutation operators is based on the previous success probability. F follows a normal distribution within a certain range, while CR performs adaptive control based on historical experience.

The reasons for selecting these seven DE variants as comparison algorithms are interpreted as follows: First, JADE and SHADE are two well-known DE variants used as comparison algorithms by numerous related researchers. Second, SaDE and jDE are selected because they have adaptive mutation or parameter control strategies. Third, DE and DE-random are used to validate the adaptive skills of DRLDE. Additionally, five recent proposed advanced optimizers are included in comparative analysis, which are as follows:

- **Dynamic Combination Differential Evolution (DCDE)[86]:** DCDE is a novel variant of DE algorithm. It adopts a dynamically combined mutation operator and a two-level parameter adaptive strategy to improve the performance of the DE algorithm, and performs well in single objective continuous optimization problems;
- **Hierarchical Structure-based Joint Operations Algorithm (HSJOA)[87]:** HSJOA is an improved joint operations algorithm that designs targeted execution strategies and algorithm structures for core operations, significantly improving the optimization performance of the original Joint Operations Algorithm (JOA);
- **Leader-Adjoint Differential Evolution (LADE)[17]:** LADE is a heuristic algorithm that uses a leader assistant model to balance exploration and utilization, effectively solving the problems of premature convergence and stagnation, and demonstrating good performance in mathematical optimization and engineering applications;
- **Nutcracker Optimization Algorithm (NOA)[88]:** NOA is a novel metaheuristic algorithm based on natural inspiration, which has shown excellent results in standard test functions, CEC test suites, and some practical engineering design problems, demonstrating strong potential in solving complex optimization problems;
- **Two-stage Differential Evolution (TDE)[89]:** TDE is a novel optimization algorithm based on differential evolution, which demonstrates good performance in real parameter single objective global optimization problems through innovative two-stage mutation strategies, parameter control schemes, and population management mechanisms.

It should be noticed that the algorithms used in our experiment are all based on the default settings recommended by the authors and those of DE variants and recent advanced optimizers are summarized in Tables 3 and 4.

4.3 Comparison results

To mitigate the impact, statistical tests are conducted. The Wilcoxon signed rank-sum hypothesis test is adopted in this study, with a significance level set at $\alpha = 5\%$. The hypothesis will be rejected if the p-value is less than or equal to 0.05, otherwise, it will be accepted. Through hypothesis testing, we can determine whether there is a significant statistical difference between the DRLDE algorithm and other algorithms. The test results are shown in the “WR” column of the tables with symbols +, −, and ≈. The symbol “+” indicates that the compared algorithm is statistically significantly better than the DRLDE algorithm on a certain problem. Similarly, “−” represents “worse”, and “≈” represents “similar”.

Good generalization performance is crucial for an excellent algorithm, meaning it should achieve robust

Table 3 Parameter configuration of the compared algorithms.

Algorithm	Operation		Control parameter	Hyperparameter
	Mutation	Crossover		
DE[62]	DE/rand/1	bin	$N = 5n$, $F = 0.5$, CR = 0.8	NA
JADE[18]	DE/current-to-pbest/1	bin	N = 30, 100, 200 when D = 10, 30, 50, respectively	$N_{\text{archive}} = N$, $\mu_F = 0.5$, $\mu_{\text{CR}} = 0.5$, c=0.1, p=0.05
CoDE[76]	DE/rand/1	bin	N = 30	F = 1.0, 0.8 CR = 0.1, 0.2, 0.9
	DE/rand/2	bin		
	DE/current-to-rand/1	ari		
jDE[84]	DE/rand/1	bin	N = 100, 100, 200 when D = 10, 30, 50, respectively	$\tau_1 = \tau_2 = 0.1$, $F_1 = 0.1$, $F_u = 0.9$
SHADE[83]	DE/current-to-pbest/1	bin	N = 100	$M_F = M_{\text{CR}} = 0.5$, H = 100
SaDE[85]	DE/rand/1	bin	N = 50, 100, 200 when D = 10, 30, 50, respectively	$p_1 = p_2 = 0.5$, $\mu_F = 0.5$, $\mu_{\text{CR}} = 0.5$
	DE/current-to-best/2	bin		

Table 4 Parameter setting of recent advanced optimizers.

Algorithm	Parameter setting
DCDE[86]	NP= $2 \cdot D$, $m = 1$, $n = 3$
HSJOA[87]	$C_{\text{nu}} = 6$, $C_{Ep} = 18$, NP= 66
LADE[17]	$\text{NP}_{\text{init}} = 18 \cdot D$, $p = 0.11$, $r^{\text{rac}} = 2.6$, $\tau_1 = \tau_2 = 0.01$, $c = 0.1$
NOA[88]	$P_{\text{rp}} = 0.2$, $P_{a2} = 0.4$, $\delta = 0.05$, NP= 25
TDE[89]	$\mu_F = 0.3$, $\mu_{\text{CR}} = 0.8$, $F \sim C(\mu_F, 0.1)$, CR$\sim N(\mu_{\text{CR}}, 0.1)$, $p = 0.25 - 0.11$, $r^{\text{rac},A} = 1.6$, $r^{\text{rac},B} = 3$, $k = 4$, NP= $25 \cdot \log D \cdot \sqrt{D}$, $n = 2 \cdot D$, $\rho = 2/3 \cdot$MAXFES

performance across various test problems. In this paper, the algorithm's overall performance is assessed using the Average Performance Score (APS)[90]. Suppose we compare m algorithms to M problems. For any two algorithms i and j, $i, j \in [1, m]$ on any problem $k \in [1, M]$, $\delta_{ij} = 0$ if A_i performs better than A_j on the k-th problem f_k with statistical significance; otherwise, $\delta_{ij} = 1$. The APS of A_i on F_k is calculated by

$$P_k(A_i) = \sum_{j \in [1, n] \setminus \{i\}} \delta_{ij} \tag{19}$$

For ease of reading, specific experimental data are presented in the form of a table in the supplementary materials.

4.3.1 Comparison results with JADE, SHADE, CoDE, jDE, SaDE, DE, and DE-random

In this section, JADE, SHADE, CoDE, jDE, SaDE, and DE are used as comparative algorithms to verify the performance of DRLDE. The test function includes the 10 and 30 dimensions of CEC2013, as well as the 10, 30, and 50 dimensions of CEC2017. The specific experimental results are presented in Table S1–S5, which are in the Electronic Supplementary Material (ESM) of the online version of this article.

The ranking results according to APS of all compared methods are shown in Tables 5 and 6. To better visualize the ranking results, the average rankings over 10-D, 30-D, and 50-D for all methods are shown in Fig. 5. From Tables S1 and S2, it can be seen that in the 10-D and 30-D problems, DRLDE outperforms other comparative algorithms on 10 and 12 problems, respectively, and its overall performance ranks second, only after SHADE (the best DE-based algorithm on CEC2013).

Similarly, as can be seen from Table 6, for the CEC2017 benchmark functions, DRLDE ranks first on the 10-D problems, with 16 problems surpassing all comparison algorithms and ranking second on the 30-D problems. Specifically, the algorithm is tested on the 50-D problems, while the DRLDE does not encounter any 50-D problems during training. From the test results, it is evident that even on the untrained 50-D problems, DRLDE still has good competitiveness compared to other state-of-the-art algorithms. The

Table 5 Average ranking based on APS on the CEC2013 benchmark suite. The best results are marked in bold.

Algorithm	D = 10	D = 30
SHADE	**1.39**	**1.00**
DRLDE	1.85	1.39
CoDE	1.75	2.10
JADE	5.14	1.71
SaDE	3.57	3.25
DE-random	2.85	5.25
jDE	2.85	3.42
DE	2.17	4.28

Table 6 Average ranking based on APS on the CEC2017 benchmark suite. The best results are marked in bold.

Algorithm	$D = 10$	$D = 30$	$D = 50$
DRLDE	**1.17**	1.10	1.89
SHADE	2.10	**0.93**	**1.69**
CoDE	1.68	2.17	2.65
DE-random	1.65	4.12	4.15
jDE	3.27	3.17	3.20
JADE	4.27	2.72	2.82
SaDE	2.27	4.24	3.89
DE	3.86	6.10	6.86

comprehensive performance of DRLDE on CEC2017 ranks first and is better than SHADE. Moreover, the idea of CoDE is similar to ours, which involves simultaneously controlling mutation strategies, F, and CR by setting parameter combinations. From the experimental results, we can see that DRLDE is better than CoDE in both test suits. This means that DRLDE has a more advantageous parameter combination and adaptation mechanism.

4.3.2 Comparison results with other recent advanced optimizers

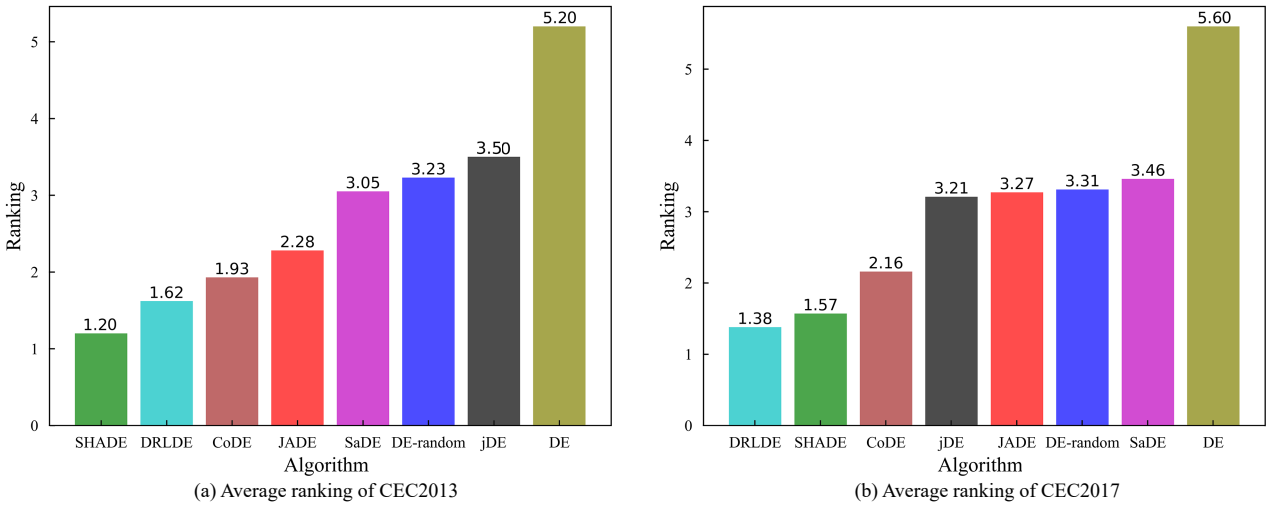
Moreover, to comprehensively demonstrate the merit of DRLDE, four advanced heuristic algorithms are employed for comparison. The experiment is conducted on 30-dimensional functions of CEC2017. The experimental results are shown in Table 7. The results show that DRLDE performs much better than three recent advanced optimizers (HSJOA, NOA, and TDE), with only DCDE and LADE perform quite close to DRLDE. From this, it can be concluded that DRLDE has strong competitiveness in both well-known DE variants and recent advanced algorithms.

4.4 Adaptation behavior analysis

To analyze the adaptive behavior of DRLDE during the evolutionary process, Fig. 6 presents a heat map of action frequency across all test functions from CEC2013, illustrating the overall trend of actions. After that, several representative problems are selected for adaptive behavior analysis, namely f3, f15, f18, f23, and f27. The reason why these problems are selected is that these functions cover uni-modal, multi-modal, and composition functions, and DRLDE's performance in these functions is far better than that of untrained agents (DE-random), so it is suitable for analyzing adaptive behavior.

In Fig. 6, each index on the y-axis represents an action. The specific action corresponding to each index can be found in Table 1. It can be observed from the heat map that for all functions, the untrained strategy (DE-random) completely randomly selects each action, so the frequency of all actions is similar; on the contrary, DRLDE has a clear tendency for the choice of actions (especially the actions of 3, 4, and 8), and for different problems, the frequency of actions is significantly different. It means that through training, the agent learned an effective strategy and can select appropriate actions according to different test functions.

Figure 7 is the action analysis diagram of F3. Figure 7a is a scatter plot that records the actions with the highest frequency selected by DRLDE and DE-random in each iteration. Figure 7b is a 3-D map of the test function. Figure 7c is the frequency chart of each



(a) Average ranking of CEC2013

(b) Average ranking of CEC2017

Fig. 5 Average ranking of different algorithms.

Table 7 Comparison of algorithms on CEC2017 benchmark problems. The optimal results of the algorithms are marked in bold.

Problem	DRLDE		HSJOA			DCDE			LADE			TDE			NOA		
	Mean	Std	Mean	Std	WR	Mean	Std	WR	Mean	Std	WR	Mean	Std	WR	Mean	Std	WR
F1	**0**	**0**	7.69×10^{3}	6.69×10^{3}	–	5.68×10^{-12}	1.70×10^{-11}	–	1.28×10^{-14}	4.26×10^{-15}	–	3.98×10^{-14}	1.53×10^{-14}	–	1.33×10^{7}	6.42×10^{6}	–
F3	**0**	**0**	3.47×10^{4}	2.41×10^{4}	–	1.62×10^{-1}	1.29×10^{-1}	–	9.09×10^{-14}	2.78×10^{-14}	–	3.47×10^{-13}	1.12×10^{-13}	–	1.65×10^{4}	5.33×10^{3}	–
F4	$\mathbf{2.83\times10^{1}}$	$\mathbf{3.07\times10^{1}}$	9.15×10^{1}	1.01×10^{1}	–	8.75×10^{1}	9.47×10^{-1}	–	3.22×10^{1}	2.98×10^{1}	–	4.15×10^{1}	4.38×10^{1}	–	8.83×10^{1}	1.50×10^{1}	–
F5	3.13×10^{1}	7.57×10	7.92×10^{1}	2.09×10^{1}	–	$\mathbf{2.03\times10^{1}}$	$\mathbf{6.42\times10}$	+	2.40×10^{1}	2.81×10	+	7.54×10^{1}	2.26×10^{1}	–	1.09×10^{2}	2.07×10^{1}	–
F6	8.66×10^{2}	1.15×10^{1}	1.44×10^{1}	6.90×10	+	$\mathbf{6.69\times10^{-8}}$	$\mathbf{1.01\times10^{-7}}$	+	3.91×10^{-3}	9.28×10^{-3}	+	1.10×10^{1}	5.89×10	+	4.78×10^{1}	9.64×10	+
F7	5.87×10^{1}	9.70×10	1.13×10^{2}	2.17×10^{1}	–	$\mathbf{5.05\times10^{1}}$	$\mathbf{7.52\times10}$	+	5.39×10^{1}	2.42×10	+	1.00×10^{2}	2.03×10^{1}	–	2.09×10^{2}	3.31×10^{1}	–
F8	3.04×10^{1}	1.03×10^{1}	7.72×10^{1}	2.23×10^{1}	–	2.40×10^{1}	3.84×10	+	$\mathbf{2.33\times10^{1}}$	$\mathbf{4.50\times10}$	+	6.01×10^{1}	8.58×10	–	1.14×10^{2}	3.16×10^{1}	–
F9	9.20×10^{1}	1.64×10^{1}	4.97×10^{2}	4.47×10^{2}	–	$\mathbf{7.33\times10^{-29}}$	$\mathbf{3.87\times10^{-30}}$	+	4.55×10^{-14}	5.57×10^{-14}	+	7.81×10^{1}	7.66×10^{1}	+	4.54×10^{3}	1.35×10^{3}	–
F10	$\mathbf{1.85\times10^{3}}$	$\mathbf{4.56\times10^{2}}$	4.29×10^{3}	3.64×10^{2}	–	1.86×10^{3}	4.63×10^{2}	≈	2.20×10^{3}	2.23×10^{2}	–	4.02×10^{3}	1.05×10^{3}	–	3.90×10^{3}	6.28×10^{2}	–
F11	$\mathbf{2.65\times10^{1}}$	$\mathbf{1.53\times10^{1}}$	9.74×10^{1}	2.17×10^{1}	–	3.97×10^{1}	2.67×10^{1}	–	3.34×10^{1}	2.76×10^{1}	–	1.27×10^{2}	5.68×10^{1}	–	2.11×10^{2}	3.18×10^{1}	–
F12	4.64×10^{3}	5.92×10^{3}	9.79×10^{5}	8.81×10^{5}	–	8.34×10^{3}	5.53×10^{3}	–	$\mathbf{4.64\times10^{3}}$	$\mathbf{2.91\times10^{3}}$	≈	1.48×10^{4}	1.52×10^{4}	–	1.53×10^{7}	1.37×10^{7}	–
F13	$\mathbf{3.40\times10^{1}}$	$\mathbf{2.31\times10^{1}}$	1.63×10^{4}	2.21×10^{4}	–	6.79×10^{3}	4.24×10^{3}	–	3.44×10^{1}	2.65×10^{1}	≈	1.08×10^{3}	8.47×10^{2}	–	3.51×10^{4}	2.50×10^{4}	–
F14	$\mathbf{2.91\times10^{1}}$	$\mathbf{4.80\times10}$	6.15×10^{4}	3.89×10^{4}	–	2.93×10^{1}	3.80×10	≈	3.64×10^{1}	6.66×10	–	2.48×10^{2}	7.38×10^{1}	–	2.82×10^{2}	4.09×10^{1}	–
F15	$\mathbf{1.14\times10^{1}}$	$\mathbf{4.11\times10}$	3.10×10^{4}	1.18×10^{4}	–	1.52×10^{1}	7.54×10	–	1.59×10^{1}	1.39×10^{1}	–	2.15×10^{2}	1.12×10^{2}	–	3.20×10^{3}	4.25×10^{3}	–
F16	$\mathbf{3.81\times10^{1}}$	$\mathbf{1.54\times10^{2}}$	9.77×10^{2}	2.18×10^{2}	–	1.88×10^{2}	1.35×10^{2}	–	3.19×10^{2}	7.81×10^{1}	–	3.12×10^{2}	1.79×10^{2}	–	1.15×10^{3}	2.19×10^{2}	–
F17	4.99×10^{1}	2.42×10	3.26×10^{2}	1.34×10^{2}	–	$\mathbf{3.15\times10^{1}}$	$\mathbf{5.12\times10}$	+	5.82×10^{1}	6.64×10	–	2.51×10^{1}	1.21×10^{2}	–	3.20×10^{2}	1.30×10^{2}	–
F18	$\mathbf{2.99\times10^{1}}$	$\mathbf{7.32\times10}$	4.84×10^{5}	4.08×10^{5}	–	3.32×10^{2}	3.96×10^{2}	–	8.32×10^{1}	9.40×10^{1}	–	1.39×10^{3}	2.84×10^{3}	–	6.17×10^{3}	6.17×10^{3}	–
F19	8.62×10^{2}	2.19×10	2.88×10^{4}	2.04×10^{4}	–	$\mathbf{1.06\times10^{1}}$	$\mathbf{2.00\times10}$	+	1.26×10^{1}	2.94×10	+	1.80×10^{2}	6.13×10^{1}	+	3.65×10^{3}	5.26×10^{3}	–
F20	$\mathbf{7.19\times10^{1}}$	$\mathbf{6.09\times10}$	4.11×10^{2}	1.35×10^{2}	–	9.90×10^{1}	5.61×10^{1}	–	9.81×10^{1}	5.26×10^{1}	–	1.44×10^{2}	7.22×10^{1}	–	3.59×10^{2}	1.21×10^{2}	–
F21	2.29×10^{2}	7.43×10	2.90×10^{2}	1.67×10^{1}	–	$\mathbf{2.21\times10^{2}}$	$\mathbf{5.58\times10}$	≈	2.25×10^{2}	3.50×10	≈	2.60×10^{2}	1.13×10^{1}	–	3.16×10^{2}	1.82×10^{1}	–
F22	1.00×10^{2}	9.81×10	3.18×10^{3}	1.83×10^{3}	–	$\mathbf{1.00\times10^{2}}$	$\mathbf{0.00\times10}$	≈	1.00×10^{2}	1.59×10^{-13}	≈	1.46×10^{3}	2.16×10^{3}	–	4.12×10^{3}	3.98×10^{2}	–
F23	3.73×10^{2}	9.47×10	4.68×10^{2}	2.41×10^{1}	–	$\mathbf{3.67\times10^{2}}$	$\mathbf{7.83\times10}$	≈	3.76×10^{2}	4.49×10	≈	4.21×10^{2}	1.21×10^{1}	–	4.72×10^{2}	1.78×10^{1}	–
F24	4.47×10^{2}	1.04×10^{1}	5.95×10^{2}	2.53×10^{1}	–	4.43×10^{2}	6.22×10	≈	$\mathbf{4.42\times10^{2}}$	$\mathbf{4.74\times10}$	≈	4.97×10^{2}	2.83×10^{1}	–	5.10×10^{2}	2.76×10^{1}	–
F25	4.80×10^{2}	5.10×10^{1}	3.88×10^{2}	1.40×10	+	$\mathbf{3.87\times10^{2}}$	$\mathbf{9.39\times10^{-2}}$	+	$\mathbf{3.87\times10^{2}}$	$\mathbf{1.78\times10^{-1}}$	+	3.89×10^{2}	2.14×10	+	3.97×10^{2}	1.85×10^{1}	+
F26	1.16×10^{3}	4.29×10^{2}	2.13×10^{3}	1.82×10^{2}	–	$\mathbf{1.09\times10^{3}}$	$\mathbf{1.16\times10^{2}}$	+	1.20×10^{3}	6.34×10^{1}	≈	1.62×10^{3}	7.19×10^{2}	–	2.48×10^{3}	2.60×10^{2}	–
F27	$\mathbf{4.92\times10^{2}}$	$\mathbf{1.03\times10^{1}}$	5.16×10^{2}	8.96×10	≈	5.01×10^{2}	3.94×10	≈	5.02×10^{2}	4.59×10	≈	5.32×10^{2}	1.93×10^{1}	–	5.42×10^{2}	9.98×10	–
F28	$\mathbf{3.15\times10^{2}}$	$\mathbf{3.70\times10^{1}}$	2.07×10^{3}	1.16×10^{3}	–	4.17×10^{2}	1.38×10^{1}	–	3.21×10^{2}	4.13×10^{1}	≈	3.85×10^{2}	7.06×10^{1}	–	4.75×10^{2}	$2.67e\times10^{1}$	–
F29	1.14×10^{3}	4.12×10^{2}	7.00×10^{2}	1.48×10^{2}	+	$\mathbf{4.67\times10^{2}}$	$\mathbf{2.05\times10^{1}}$	+	4.83×10^{2}	2.03×10^{1}	+	7.10×10^{2}	1.15×10^{2}	+	1.23×10^{3}	1.28×10^{2}	–
F30	$\mathbf{2.08\times10^{3}}$	$\mathbf{6.60\times10^{1}}$	1.17×10^{5}	1.60×10^{5}	–	3.79×10^{3}	6.02×10^{2}	–	2.14×10^{3}	8.26×10^{1}	≈	5.27×10^{3}	4.11×10^{3}	–	3.12×10^{5}	3.19×10^{5}	–
–/≈/+	–		25/1/3			12/7/10			11/10/8			24/0/5			27/0/2		

action in each iteration in 51 independent runs, in which actions 3, 4, and 8 are represented by a line chart, while other actions are presented in the form of scatter chart (the interpretation of Figs. 8–11 is the same as that of Fig. 7).

F3 is a rotated bent-cigar function. It can be seen from the 3-D map (Fig. 7b) that its landscape is smooth but has a narrow ridge. As mentioned earlier, "DE/current-to-rand/1" is propitious to solve rotated problems, so the strategy is selected most frequently (Fig. 7a). In addition, "F = 1.0, CR = 0.9" is selected to preserve population diversity, and to find a better solution in the narrow ridge. We can find that in the initial and final stage of the solution, "DE/current-to-rand/1, F = 0.8, CR = 0.2" has the highest frequency, while "DE/current-to-pbest/1, F = 1.0, CR = 0.9" has a higher frequency in the middle stage. It means that it is very important to preserve population diversity in the middle stage of the evolutionary process.

F15 is a rotated Schwefel's function with a multi-modal landscape, and it has a large number of local optimums, and the location of the sub-optimal solution is far from the global optimum. Figure 8 illustrates that in the initial stage, the frequency of actions 4, 8, and 3 is close, while the frequency of action 4 in the middle stage is much higher than that of other actions. It is because "DE/current-to-pbest/1" prefers to search in local regions and is more effective in dealing with multi-modal optimization problems. Similarly, in the medium term, "F = 1.0, CR= 0.9" is selected more to preserve population diversity.

F18 is a multi-modal function with continuous everywhere yet differentiable nowhere property. It can be seen from Fig. 9 that action 8 has the highest

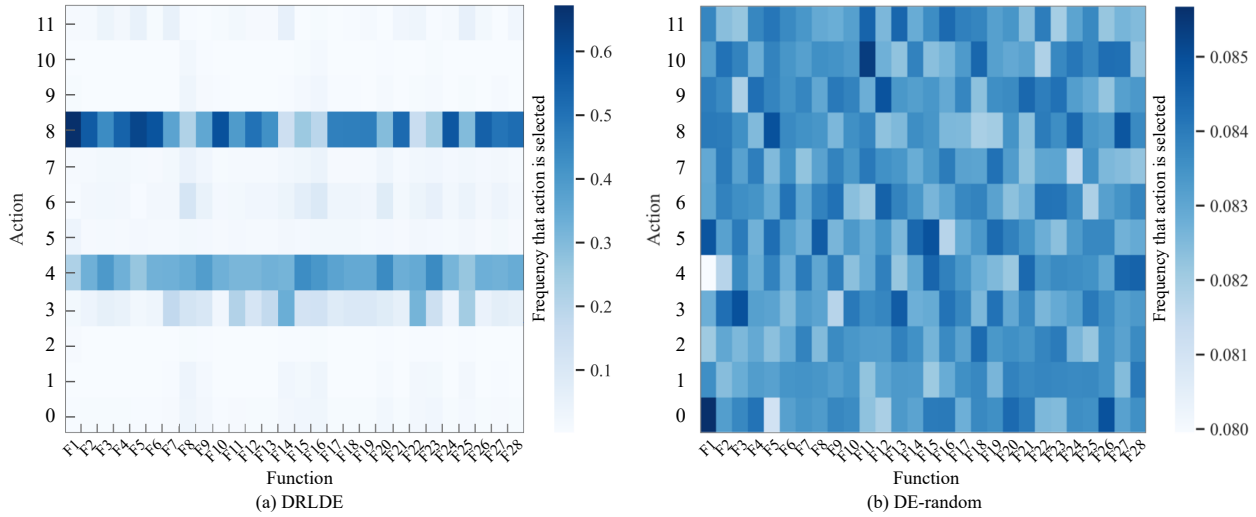


Fig. 6 Heatmap of action frequency.

frequency in the initial and final periods. Similarly, the frequency of action 4 increased significantly in the middle period to maintain population diversity.

F23 is a composition function composed of three basic functions. It can be seen from Fig. 10 that the 3-D map of F23 is very similar to that of F15, so we can think that they have similar properties. Coincidentally, a surprising phenomenon is that when solving the two functions F15 and F23, the frequency of each action is almost the same in each period (Figs. 8c and 10c). Therefore, we have reason to believe that DRLDE will use similar strategies to select appropriate actions for functions with similar properties.

F27 is also a composition function composed of five basic functions, but the selected action trend differs from F23. This is because the 3-D map of F27 is much more different from F23. Similar to the other functions, "$F = 1.0$, CR = 0.9" will be more selected in the middle stage to maintain population diversity.

On the whole, the most frequent actions are actions 3, 4, and 8, especially actions 4 and 8, which may be because the combination of these two actions can not only explore the search space to the maximum extent but also maintain a fast convergence speed. We can also find a rule: in the initial stage and final stage of the solution, the frequency of the 8th action is the highest, while in the middle stage, the frequency of action 4 is the highest. This is in line with our previous assumption: action 8 can better balance and accelerate the population convergence and preserve the diversity of the population, while action 4 can explore the search space to the maximum in the middle stage and escape from the local optimum. In addition, it can be seen from the examples of F15 and F23 that DRLDE will use similar strategies to select actions for functions with similar landscapes. For functions with different properties, DRLDE will have different strategies for selecting actions. In any case, we can think that the proposed DRLDE algorithm has an effective adaptive mechanism for different optimization problems.

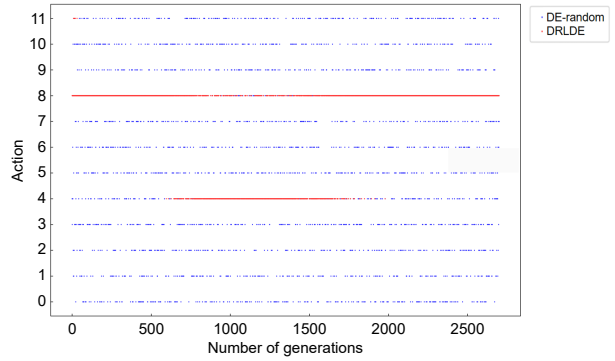
4.5 Contributions of the deep reinforcement learning

The adaptive behavior analysis in Section 4.4 shows that actions 3, 4, and 8 are most frequently selected. To determine whether a single action can yield effective results, DRLDE is compared with DE variants that employ a single strategy. DE-3 represents a single strategy DE with action 3. The same naming convention applies to DE-4 and DE-8. As described in Section 4.3, the average ranking result is shown in Fig. 4. The numerical results are presented in Table S6.

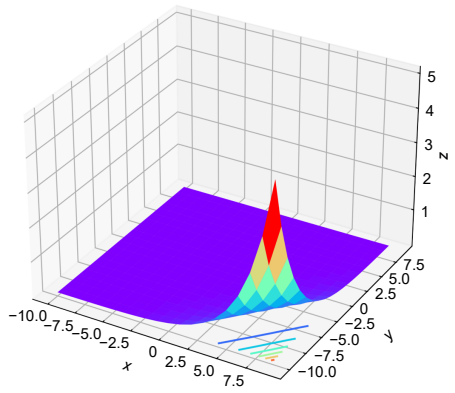
It can be seen from the experimental results that DRLDE outperforms all three single strategy-based methods. This suggests that with the aid of deep reinforcement learning, DRLDE has successfully learned to select the most suitable strategies at optimal stages.

4.6 Computational complexity analysis

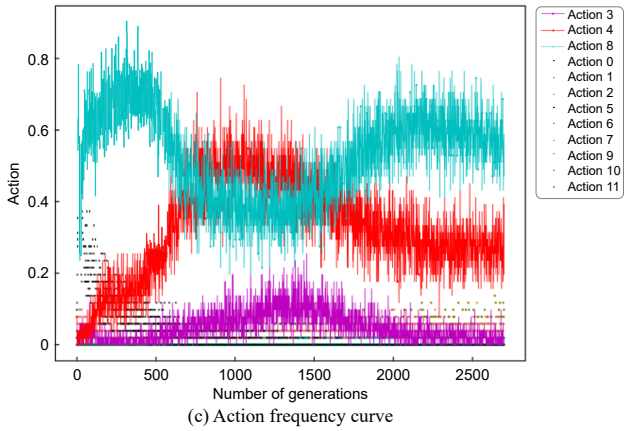
The time complexity is calculated based on the method shown in Ref. [82] and compared to five other state-of-the-art metaheuristic algorithms, that is, LADE, DCDE, NOA, HSJOA, and TDE. The comparison result is presented in Table 8. All the results are obtained by running on the same computing platform.



(a) Actions with the highest selection frequency in each generation

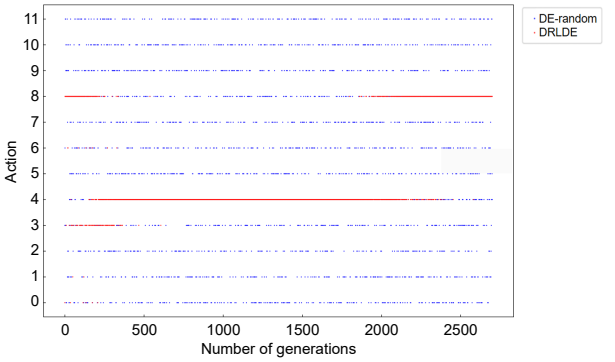


(b) 3-D map

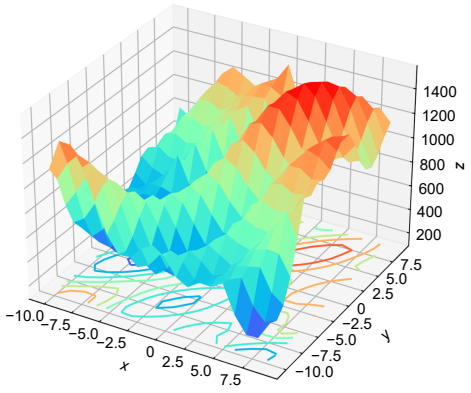


(c) Action frequency curve

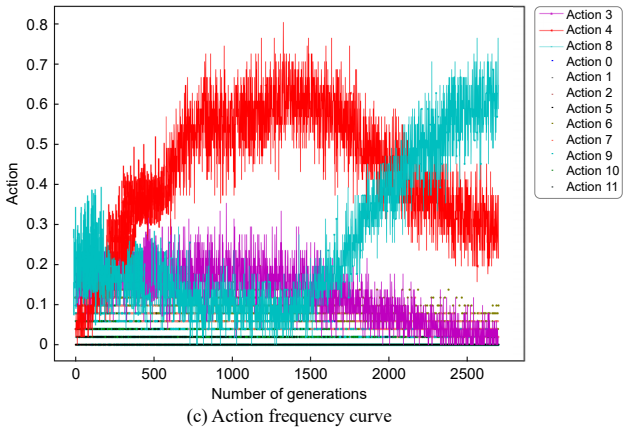
Fig. 7 Action analysis diagram of F3.



(a) Actions with the highest selection frequency in each generation



(b) 3-D map



(c) Action frequency curve

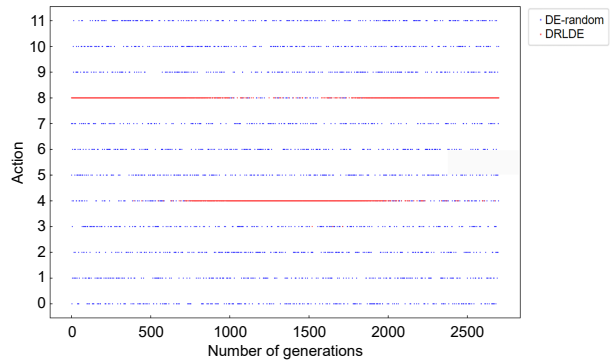
Fig. 8 Action analysis diagram of F15.

From the results, It can be seen that DRLDE runs slightly slower than TDE, but runs faster than other algorithms, especially on higher dimensional problems (30-D and 50-D). Overall, DRLDE is an efficient optimization algorithm with reasonable time costs, especially for high dimensional problems.

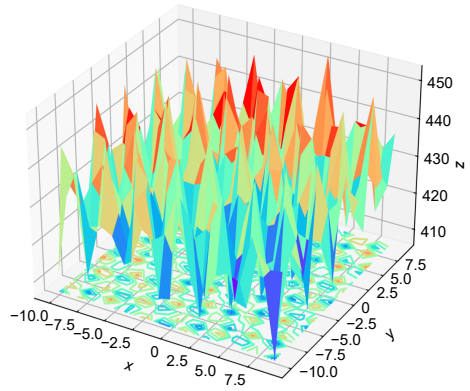
5 Real-World Application on Rover Trajectory Planning

To assess how well the trained DRLDE algorithm performs on real-world problems that it has not encountered before, we assess its performance on a 60-D rover trajectory planning problem. The results are compared with those of other advanced algorithms. Furthermore, classical DE serves as a benchmark for evaluating the performance of alternative algorithms. It is important to note that all parameter configurations remain consistent with previous experiments. Each optimization problem is repeated 10 times independently.

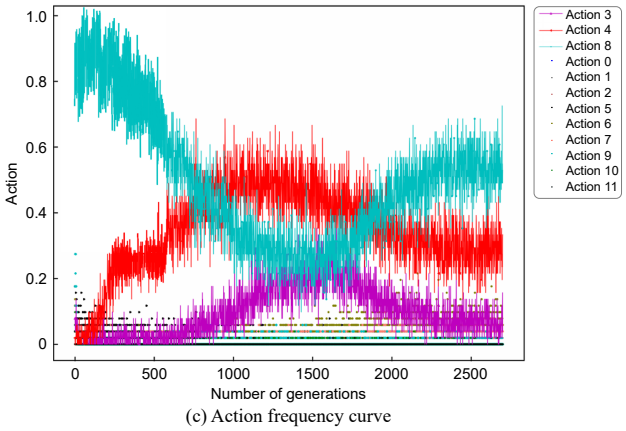
This 60-D trajectory optimization problem in 2D is from Ref. [91], to obtain the optimal trajectory from the starting point to the endpoint while minimizing the cost function. More details can be found in this



(a) Actions with the highest selection frequency in each generation

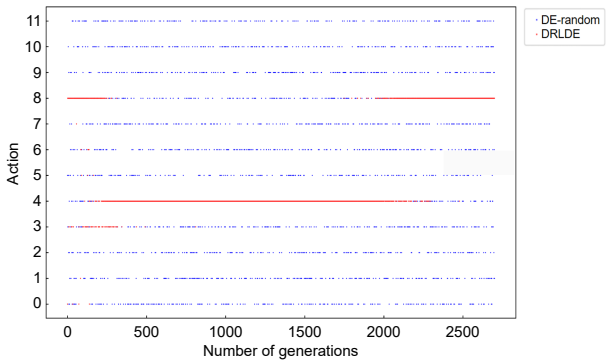


(b) 3-D map

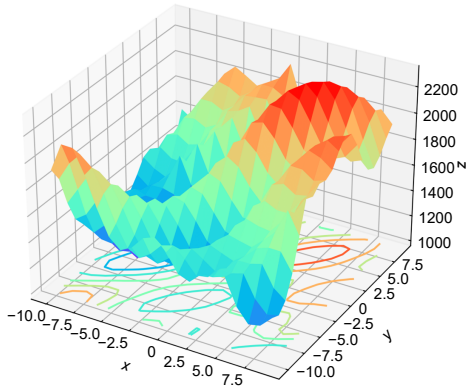


(c) Action frequency curve

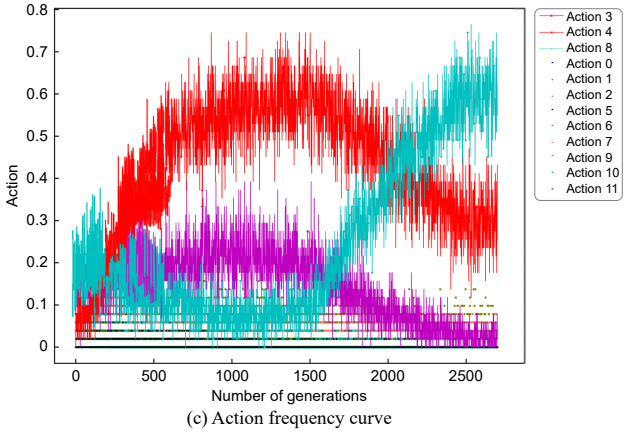
Fig. 9 Action analysis diagram of F18.



(a) Actions with the highest selection frequency in each generation



(b) 3-D map



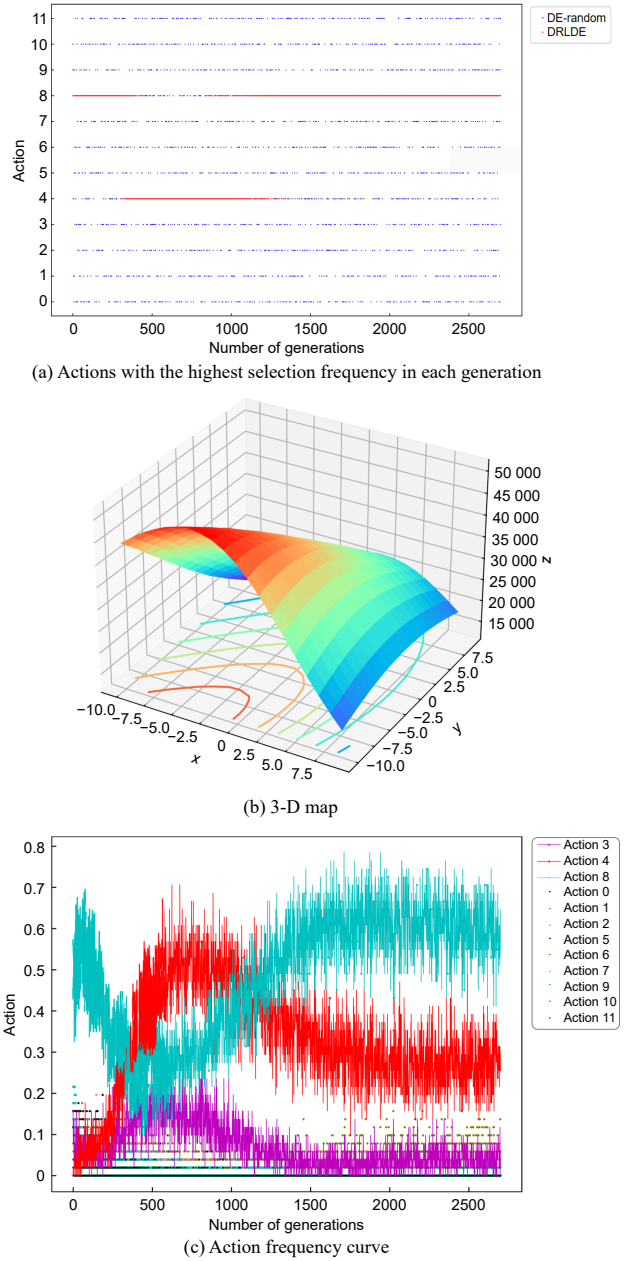
(c) Action frequency curve

Fig. 10 Action analysis diagram of F23.

literature. Simply put, this problem has characteristics of discontinuity, concavity, and non-smoothness. The experimental results are shown in Fig. 13.

The rising curve of reward shows that the advantages of DRLDE begin to appear after the number of evaluations reached 100 000. DRLDE does not have a very high convergence rate at the initial stage of the evolutionary process (compared to CoDE and SHADE), which means that DRLDE will spend more resources in the initial stage to explore the search space to escape local optimization. It should be noted that the agent has not been trained on the 60-D problem, which means that the problem is completely unknown to DRLDE, but DRLDE can still perform well. SHADE performs well in previous numerical experiments, but in this 60-D real-world problem, its performance is poor (similar to JADE's performance). The reason for this result may be that SHADE's design does not utilize the features of the problem, while DRLDE uses the FL method to extract the features of the problem. From this we can conclude that DRLDE, which has a good performance in both numerical experiments and real-world problems, has higher robustness.

To summarize, DRLDE ranks first in the 60-D rover



(a) Actions with the highest selection frequency in each generation

(b) 3-D map

(c) Action frequency curve

Fig. 11 Action analysis diagram of F27.

trajectory planning problem. It verifies that the proposed DRLDE algorithm has strong robustness and applicability, not only having good performance in real-parameter optimization test suites, but also achieving good results in real-world problems.

6 Discussion

In this research, DRL is utilized as the main method to dynamically choose mutation strategies and control parameters based on optimization experiences. To our knowledge, there is no existing research on adapting the DE mutation strategy and its parameters through

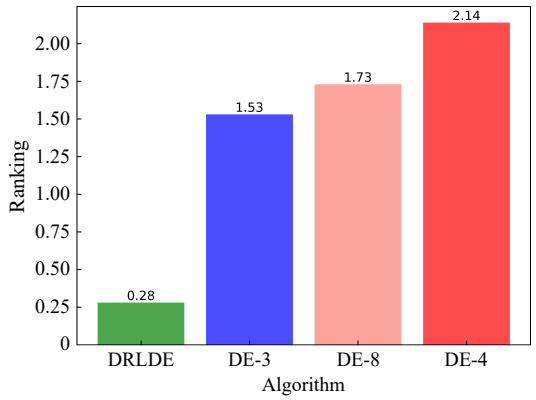


Fig. 12 Average ranking of compared algorithms.

learning from optimization experiences. On the other hand, from the aspect of the discrete actions of our DRLDE, it looks similar to CoDE[76], which also contains a few mutation and control parameter combinations. However, the differences between DRLDE and CoDE are clear as follows:

(1) After training, DRLDE selects the corresponding mutation strategies and control parameters through the saved policy model, so for each individual, only one mutation operation will be performed during each iteration. On the contrary, each individual of CoDE undergoes three mutation operations during each iteration, and the control parameters are randomly selected.

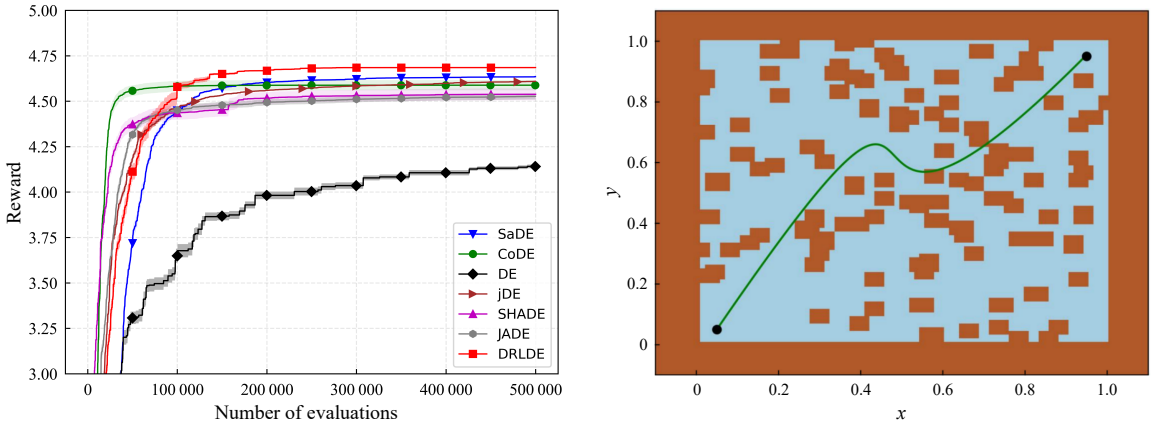
(2) The mutation strategy set of DRLDE is different from that of CoDE. DRLDE comprehensively considered the characteristics of each mutation strategy, combined with advanced research, and selected four mutation strategies with different advantages, while CoDE only used the three original strategies of "DE/rand/1", "DE/rand/2", and "DE/current-to-rand/1".

(3) The goal of CoDE is to fully utilize the experience of researchers, and its results largely depend on parameter combination settings. However, DRLDE can not only fully utilize the experience of researchers, but also intelligently learn parameter combination methods suitable for different problems through the DRL algorithm, thereby further improving algorithm performance.

The proposed DRLDE algorithm is effective due to its joint adaptation of both mutation strategies and control parameters, which ensures a comprehensive and adaptive optimization process. By utilizing distributed PPO and training on a large set of 750 problem instances, DRLDE leverages advanced DRL

Table 8 Runtime consumption comparison of DRLDE with other five algorithms.

D	T_0 (s)	T_1 (s)	DRLDE		LADE		DCDE		NOA		HSJOA		TDE	
			$\hat{T}_2$ (s)	$(\hat{T}_2-T_1)/T_0$	$\hat{T}_2$ (s)	$(\hat{T}_2-T_1)/T_0$	$\hat{T}_2$ (s)	$(\hat{T}_2-T_1)/T_0$	$\hat{T}_2$ (s)	$(\hat{T}_2-T_1)/T_0$	$\hat{T}_2$ (s)	$(\hat{T}_2-T_1)/T_0$	$\hat{T}_2$ (s)	$(\hat{T}_2-T_1)/T_0$
10	3.12	2.05	16.28	4.56	25.79	7.61	25.88	7.64	13.69	3.73	10.39	2.67	14.82	4.09
30	3.12	2.37	16.46	4.52	28.62	8.42	37.68	11.33	17.75	4.93	16.97	4.69	15.92	4.35
50	3.12	2.62	16.61	4.48	30.78	9.03	49.87	15.16	22.10	6.25	22.65	6.42	15.93	4.27



(a) Comparison of convergence curves (b) Optimized trajectory

Fig. 13 Results on 60-D rover trajectory planning.

techniques known for their stability and efficiency, enabling dynamic adjustments in the optimization process. Additionally, the integration of fitness landscape analysis into state representations provides a deep understanding of the optimization landscape, allowing for more informed decision-making.

7 Conclusion and Future Work

In this study, a joint adaptation framework for mutation strategy and control parameters in DE, i.e., DRLDE, is established, harnessing the capabilities of DRL. The distributed proximal policy optimization algorithm is utilized to learn the optimal policies. For enhancing the guidance of the optimization process, outcomes from fitness landscape analyses are integrated into the state representations of the DRL scheme. Addressing the need for diverse training scenarios, the BBOB test suite is employed, leveraging its ability to generate a large number of problem instances.

The performance of the proposed DRLDE is validated through testing on unseen problems from CEC2013 and CEC2017 test suites. The results are also compared against several other state-of-the-art methods. Experimental results show that the performance of DRLDE is competitive with the state-of-the-art methods on the test problems. Adaptation behavior analysis of DRLDE shows that it successfully learns different policies for different problems and even for different stages within the same problem. Finally, the trained DRLDE is applied to a complex 60-D real-world problem of rover trajectory planning, and it ranks first among other excellent comparison algorithms over 10 repeated runs. The excellent performance of DRLDE on untrained high dimensional benchmark functions and 60-D real-world problems proves its high robustness.

A limitation observed in the present study pertains to the high computational intensity of the training process, even with the deployment of 50 agents. As a prospective direction, more data-efficient deep RL algorithms will be applied to improve the sample efficiency of training. In addition, the DRLDE shows an amazing adaptation capability both in problems and optimization stages, it is worth studying the intelligent adaptation behavior learned by the agent further, which may help us better understand the working mechanism behind the evolution operations. Finally, this study proposes a universal framework that is also applicable to other evolutionary-based methods. Deploying this framework to other methods through targeted adjustments is also a worthwhile research direction.

Acknowledgment

This work was supported by the National Natural Science

Foundation of China (No. 52105244), the Open Foundation of the State Key Laboratory of Fluid Power and Mechatronic Systems (No. GZKF-202425), the Entrepreneurship and Innovation Support Plan of Chongqing for Returned Overseas Scholars (No. cx2023085), and the Independent Research Project-Key Program from the State Key Laboratory of Mechanical Transmission for Advanced Equipment (No. SKLMT-ZZKT-2024Z09).

Electronic Supplementary Material

Supplementary materials including Tables S1-S7 are available in the online version of this article at https://doi.org/10.26599/TST.2024.9010185.

References

[1] Y. Song, X. Cai, X. Zhou, B. Zhang, H. Chen, Y. Li, W. Deng, and W. Deng, Dynamic hybrid mechanism-based differential evolution algorithm and its application, *Expert Syst. Appl.*, vol. 213, p. 118834, 2023.

[2] M. K. Kar, S. Kumar, A. K. Singh, and S. Panigrahi, Reactive power management by using a modified differential evolution algorithm, *Optim. Control Appl. Methods*, vol. 44, no. 2, pp. 967–986, 2023.

[3] H. Pu, H. Cheng, G. Wang, J. Ma, J. Zhao, R. Bai, J. Luo, and J. Yi, Dexterous workspace optimization for a six degree-of-freedom parallel manipulator based on surrogate-assisted constrained differential evolution, *Appl. Soft Comput.*, vol. 139, p. 110228, 2023.

[4] C. Li, G. Sun, L. Deng, L. Qiao, and G. Yang, A population state evaluation-based improvement framework for differential evolution, *Inf. Sci.*, vol. 629, pp. 15–38, 2023.

[5] A. W. Mohamed, A. A. Hadi, P. Agrawal, K. M. Sallam, and A. K. Mohamed, Gaining-sharing knowledge based algorithm with adaptive parameters hybrid with IMODE algorithm for solving CEC 2021 benchmark problems, in *Proc. 2021 IEEE Congress on Evolutionary Computation* (*CEC*), Kraków, Poland, 2021, pp. 841–848.

[6] W. Deng, S. Shang, X. Cai, H. Zhao, Y. Zhou, H. Chen, and W. Deng, Quantum differential evolution with cooperative coevolution framework and hybrid mutation strategy for large scale optimization, *Knowledge-Based Syst.*, vol. 224, p. 107080, 2021.

[7] O. Hachana and A. A. El-Fergany, Efficient PEM fuel cells parameters identification using hybrid artificial bee colony differential evolution optimizer, *Energy*, vol. 250, p. 123830, 2022.

[8] D. Ustun, A. Toktas, U. Erkan, and A. Akdagli, Modified artificial bee colony algorithm with differential evolution to enhance precision and convergence performance, *Expert Syst. Appl.*, vol. 198, p. 116930, 2022.

[9] C. Li, L. Deng, L. Qiao, and L. Zhang, An efficient differential evolution algorithm based on orthogonal learning and elites local search mechanisms for numerical optimization, *Knowledge-Based Syst.*, vol. 235, pp. 107636, 2022.

[10] X. Yu, N. Jiang, X. Wang, and M. Li, A hybrid algorithm based on grey wolf optimizer and differential evolution for UAV path planning, *Expert Syst. Appl.*, vol. 215, p. 119327, 2023.

[11] C. Huang, X. Zhou, X. Ran, J. Wang, H. Chen, and W. Deng, Adaptive cylinder vector particle swarm optimization with differential evolution for UAV path planning, *Eng. Appl. Artif. Intell.*, vol. 121, p. 105942, 2023.

[12] Y. Li, T. Han, S. Tang, C. Huang, H. Zhou, and Y. Wang, An improved differential evolution by hybridizing with estimation-of-distribution algorithm, *Inf. Sci.*, vol. 619, pp. 439–456, 2023.

[13] X. F. Liu, Z. H. Zhan, Y. Lin, W. N. Chen, Y. J. Gong, T. L. Gu, H. Q. Yuan, and J. Zhang, Historical and heuristic-based adaptive differential evolution, *IEEE Trans. Syst. Man Cybern.*: *Syst.*, vol. 49, no. 12, pp. 2623–2635, 2019.

[14] W. Yi, C. Yi, Y. Chen, Z. Pei, and J. Lu, Adaptive differential evolution with ensembling operators for continuous optimization problems, *Swarm Evol. Comput.*, vol. 69, p. 100994, 2022.

[15] Q. Yang, S. Yuan, H. Gao, and W. Zhang, Differential evolution with migration mechanism and information reutilization for global optimization, *Expert Syst. Appl.*, vol. 238, p. 122076, 2024.

[16] H. Chen, S. Li, X. Li, Y. Zhao, and J. Dong, A hybrid adaptive differential evolution based on Gaussian tail mutation, *Eng. Appl. Artif. Intell.*, vol. 119, p. 105739, 2023.

[17] Y. Li, S. Wang, H. Yang, H. Chen, and B. Yang, Enhancing differential evolution algorithm using leader-adjoint populations, *Inf. Sci.*, vol. 622, pp. 235–268, 2023.

[18] J. Zhang and A. C. Sanderson, JADE: Adaptive differential evolution with optional external archive, *IEEE Trans. Evol. Comput*., vol. 13, no. 5, pp. 945–958, 2009.

[19] V. Stanovov, S. Akhmedova, and E. Semenkin, LSHADE algorithm with rank-based selective pressure strategy for solving CEC 2017 benchmark problems, in *Proc. 2018 IEEE Congress on Evolutionary Computation* (*CEC*), Rio de Janeiro, Brazil, 2018, pp. 1–8.

[20] A. W. Mohamed, A. A. Hadi, and K. M. Jambi, Novel mutation strategy for enhancing SHADE and LSHADE algorithms for global numerical optimization, *Swarm Evol. Comput.*, vol. 50, p. 100455, 2019.

[21] M. H. Nadimi-Shahraki, S. Taghian, S. Mirjalili, and H. Faris, MTDE: An effective multi-trial vector-based differential evolution algorithm and its applications for engineering design problems, *Appl. Soft Comput*., vol. 97, p. 106761, 2020.

[22] W. Deng, H. Liu, J. Xu, H. Zhao, and Y. Song, An improved quantum-inspired differential evolution algorithm for deep belief network, *IEEE Trans. Instrum. Meas.*, vol. 69, no. 10, pp. 7319–7327, 2020.

[23] K. M. Sallam, S. M. Elsayed, R. K. Chakrabortty, and M. J. Ryan, Improved multi-operator differential evolution algorithm for solving unconstrained problems, in *Proc. 2020 IEEE Congress on Evolutionary Computation* (*CEC*), Glasgow, UK, 2020, pp. 1–8.

[24] Z. Zeng, M. Zhang, T. Chen, and Z. Hong, A new selection operator for differential evolution algorithm,

Knowledge-Based Syst., vol. 226, p. 107150, 2021.
[25] A. Kumar, P. P. Biswas, and P. N. Suganthan, Differential evolution with orthogonal array-based initialization and a novel selection strategy, *Swarm Evol. Comput.*, vol. 68, p. 101010, 2022.
[26] M. Duan, C. Yu, S. Wang, and B. Li, A differential evolution algorithm with a superior-inferior mutation scheme, *Soft Comput.*, vol. 27, no. 23, pp. 17657–17686, 2023.
[27] G. Karafotias, M. Hoogendoorn, and A. E. Eiben, Parameter control in evolutionary algorithms: Trends and challenges, *IEEE Trans. Evol. Comput.*, vol. 19, no. 2, pp. 167–187, 2015.
[28] G. S. Hao, M. H. Lim, Y. S. Ong, H. Huang, and G. G. Wang, Domination landscape in evolutionary algorithms and its applications, *Soft Comput.*, vol. 23, pp. 3563–3570, 2019.
[29] Y. Wang, Z. Liu, and G. G. Wang, Improved differential evolution using two-stage mutation strategy for multimodal multi-objective optimization, *Swarm Evol. Comput.*, vol. 78, p. 101232, 2023.
[30] E. N. Dragoi and V. Dafinescu, Parameter control and hybridization techniques in differential evolution: A survey, *Artif. Intell. Rev.*, vol. 45, no. 4, pp. 447–470, 2016.
[31] A. K. Qin, V. L. Huang, and P. N. Suganthan, Differential evolution algorithm with strategy adaptation for global numerical optimization, *IEEE Trans. Evol. Comput.*, vol. 13, no. 2, pp. 398–417, 2009.
[32] A. W. Mohamed and A. K. Mohamed, Adaptive guided differential evolution algorithm with novel mutation for numerical optimization, *Int. J. Mach. Learn. Cybern.*, vol. 10, no. 2, pp. 253–277, 2019.
[33] A. W. Mohamed and P. N. Suganthan, Real-parameter unconstrained optimization based on enhanced fitness-adaptive differential evolution algorithm with novel mutation, *Soft Comput.*, vol. 22, no. 10, pp. 3215–3235, 2018.
[34] N. H. Awad, M. Z. Ali, and P. N. Suganthan, Ensemble sinusoidal differential covariance matrix adaptation with Euclidean neighborhood for solving CEC2017 benchmark problems, in *Proc. 2017 IEEE Congress on Evolutionary Computation* (*CEC*), Donostia, Spain, 2017, pp. 372–379.
[35] J. Brest, M. S. Maučec, and B. Bošković, Single objective real-parameter optimization: Algorithm jSO, in *Proc. 2017 IEEE Congress on Evolutionary Computation* (*CEC*), Donostia, Spain, 2017, pp. 1311–1318.
[36] Z. Zeng, M. Zhang, H. Zhang, and Z. Hong, Improved differential evolution algorithm based on the sawtooth-linear population size adaptive method, *Inf. Sci.*, vol. 608, pp. 1045–1071, 2022.
[37] X. Liu, J. Sun, Q. Zhang, Z. Wang, and Z. Xu, Learning to learn evolutionary algorithm: A learnable differential evolution, *IEEE Trans. Emerg. Top. Computat. Intell.*, vol. 7, no. 6, pp. 1605–1620, 2023.
[38] Z. Cao, Z. Wang, Y. Fu, H. Jia, and F. Tian, An adaptive differential evolution framework based on population feature information, *Inf. Sci.*, vol. 608, pp. 1416–1440, 2022.
[39] H. Zhang, J. Sun, Z. Xu, and J. Shi, Learning unified mutation operator for differential evolution by natural evolution strategies, *Inf. Sci.*, vol. 632, pp. 594–616, 2023.
[40] Z. Meng, Dimension improvements based adaptation of control parameters in differential evolution: A fitness-value-independent approach, *Expert Syst. Appl.*, vol. 223, p. 119848, 2023.
[41] M. Zhang, Y. Han, S. Chen, M. Liu, Z. He, and N. Pan, A multi-strategy improved differential evolution algorithm for UAV 3D trajectory planning in complex mountainous environments, *Eng. Appl. Artif. Intell.*, vol. 125, p. 106672, 2023.
[42] M. M. Drugan, Reinforcement learning versus evolutionary computation: A survey on hybrid algorithms, *Swarm Evol. Comput.*, vol. 44, pp. 228–246, 2019.
[43] T. H. F. de Oliveira, L. P. de Souza Medeiros, A. D. D. Neto, and J. D. Melo, Q-Managed: A new algorithm for a multiobjective reinforcement learning, *Expert Syst. Appl.*, vol. 168, p. 114228, 2021.
[44] Y. Song, Y. Wu, Y. Guo, R. Yan, P. N. Suganthan, Y. Zhang, W. Pedrycz, S. Das, R. Mallipeddi, O. S. Ajani, et al., Reinforcement learning-assisted evolutionary algorithm: A survey and research opportunities, *Swarm Evol. Comput.*, vol. 86, p. 101517, 2024.
[45] M. G. P. de Lacerda, L. F. De Araujo Pessoa, F. B. De Lima Neto, T. B. Ludermir, and H. Kuchen, A systematic literature review on general parameter control for evolutionary and swarm-based algorithms, *Swarm Evol. Comput.*, vol. 60, p. 100777, 2021.
[46] M. Sharma, A. Komninos, M. López-Ibáñez, and D. Kazakov, Deep reinforcement learning based parameter control in differential evolution, in *Proc. Genetic and Evolutionary Computation Conf.*, Prague, Czech Republic, 2019, pp. 709–717.
[47] Y. Tian, X. Li, H. Ma, X. Zhang, K. Tan, and Y. Jin, Deep reinforcement learning based adaptive operator selection for evolutionary multi-objective optimization, *IEEE Trans. Emerg. Top. Computat. Intell.*, vol. 7, no. 4, pp. 1051–1064, 2023.
[48] Z. Tan and K. Li, Differential evolution with mixed mutation strategy based on deep reinforcement learning, *Appl. Soft Comput.*, vol. 111, p. 107678, 2021.
[49] L. Peng, Z. Yuan, G. Dai, M. Wang, and Z. Tang, Reinforcement learning-based hybrid differential evolution for global optimization of interplanetary trajectory design, *Swarm Evol. Comput.*, vol. 81, p. 101351, 2023.
[50] J. Sun, X. Liu, T. Back, and Z. Xu, Learning adaptive differential evolution algorithm from optimization experiences by policy gradient, *IEEE Trans. Evol. Comput.*, vol. 25, no. 4, pp. 666–680, 2021.
[51] Z. Xu, G. Han, L. Liu, M. Martinez-Garcia, and Z. Wang, Multi-energy scheduling of an industrial integrated energy system by reinforcement learning-based differential evolution, *IEEE Trans. Green Commun. Netw.*, vol. 5, no. 3, pp. 1077–1090, 2021.
[52] G. M. C. Leite, S. Jiménez-Fernández, S. Salcedo-Sanz, C. G. Marcelino, and C. E. Pedreira, Solving an energy resource management problem with a novel multi-objective evolutionary reinforcement learning method, *Knowledge-Based Syst.*, vol. 280, p. 111027, 2023.
[53] T. N. Huynh, D. T. T. Do, and J. Lee, Q-learning-based parameter control in differential evolution for structural

optimization, *Appl. Soft Comput.*, vol. 107, p. 107464, 2021.
[54] F. Liu, Q. Liu, Q. Tao, Y. Huang, D. Li, and D. Sidorov, Deep reinforcement learning based energy storage management strategy considering prediction intervals of wind power, *Int. J. Electr. Power Energy Syst.*, vol. 145, p. 108608, 2023.
[55] Z. Hu, W. Gong, and S. Li, Reinforcement learning-based differential evolution for parameters extraction of photovoltaic models, *Energy Rep.*, vol. 7, pp. 916–928, 2021.
[56] T. Li, Y. Meng, and L. Tang, Scheduling of continuous annealing with a multi-objective differential evolution algorithm based on deep reinforcement learning, *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 2, pp. 1767–1780, 2024.
[57] N. Mughees, M. H. Jaffery, A. Mughees, E. A. Ansari, and A. Mughees, Reinforcement learning-based composite differential evolution for integrated demand response scheme in industrial microgrids, *Appl. Energy*, vol. 342, p. 121150, 2023.
[58] R. Chen, B. Yang, S. Li, S. Wang, and Q. Cheng, An effective multi-population grey wolf optimizer based on reinforcement learning for flow shop scheduling problem with multi-machine collaboration, *Comput. Ind. Eng.*, vol. 162, p. 107738, 2021.
[59] Y. Du, J. Li, X. Chen, P. Duan, and Q. Pan, Knowledge-based reinforcement learning and estimation of distribution algorithm for flexible job shop scheduling problem, *IEEE Trans. Emerg. Top. Computat. Intell.*, vol. 7, no. 4, pp. 1036–1050, 2023.
[60] X. Zhang, S. Xia, X. Li, and T. Zhang, Multi-objective particle swarm optimization with multi-mode collaboration based on reinforcement learning for path planning of unmanned air vehicles, *Knowledge-Based Syst.*, vol. 250, p. 109075, 2022.
[61] Y. Cui, W. Hu, and A. Rahmani, A reinforcement learning based artificial bee colony algorithm with application in robot path planning, *Expert Syst. Appl.*, vol. 203, p. 117389, 2022.
[62] R. Storn and K. Price, Differential evolution–a simple and efficient heuristic for global optimization over continuous spaces, *J. Glob. Optim.*, vol. 11, no. 4, pp. 341–359, 1997.
[63] K. V. Price, R. M. Storn, and J. A. Lampinen, *Differential Evolution*: *A Practical Approach to Global Optimization*. Berlin, Germany: Springer-Verlag, 2005.
[64] Z. Meng, Y. Chen, X. Li, and F. Lin, PADE-NPC: Parameter adaptive differential evolution with novel parameter control for single-objective optimization, *IEEE Access*, vol. 8, pp. 139460–139478, 2020.
[65] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, Continuous control with deep reinforcement learning, arXiv preprint arXiv: 1509.02971, 2015.
[66] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, Proximal policy optimization algorithms, arXiv preprint arXiv: 1707.06347, 2017.
[67] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, in *Proc. 35th Int. Conf. Machine Learning*, Stockholm, Sweden, 2018, pp. 1861–1870.
[68] S. Wright, The roles of mutation, inbreeding, crossbreeding, and selection in evolution, in *Proc. 6th Int. Congress of Genetics*, Ithaca, NY, USA, 1932, pp. 356–366.
[69] T. T. Zhang, G. S. Hao, M. H. Lim, F. Gu, and X. Wang, A deep hybrid transfer learning-based evolutionary algorithm and its application in the optimization of high-order problems, *Soft Comput.*, vol. 27, no. 14, pp. 9661–9672, 2023.
[70] P. Merz, Advanced fitness landscape analysis and the performance of memetic algorithms, *Evol. Comput.*, vol. 12, no. 3, pp. 303–325, 2004.
[71] T. Jones and S. Forrest, Fitness distance correlation as a measure of problem difficulty for genetic algorithms, in *Proc. 6th Int. Conf. Genetic Algorithms*, Pittsburgh, PA, USA, 1995, pp. 184–192.
[72] M. Tomassini, L. Vanneschi, P. Collard, and M. Clergue, A study of fitness distance correlation as a difficulty measure in genetic programming, *Evolut. Comput.*, vol. 13, no. 2, pp. 213–239, 2005.
[73] S. Das, A. Abraham, U. K. Chakraborty, and A. Konar, Differential evolution using a neighborhood-based mutation operator, *IEEE Trans. Evol. Comput.*, vol. 13, no. 3, pp. 526–553, 2009.
[74] R. Mallipeddi, P. N. Suganthan, Q. K. Pan, and M. F. Tasgetiren, Differential evolution algorithm with ensemble of parameters and mutation strategies, *Appl. Soft Comput.*, vol. 11, no. 2, pp. 1679–1696, 2011.
[75] R. Gämperle, S. D. Müller, and P. Koumoutsakos, A parameter study for differential evolution, in *Proc. Intelligent Systems, Fuzzy Systems, Evolutionary Computation*, Interlaken, Switzerland, 2002, pp. 293–298.
[76] Y. Wang, Z. Cai, and Q. Zhang, Differential evolution with composite trial vector generation strategies and control parameters, *IEEE Trans. Evol. Comput.*, vol. 15, no. 1, pp. 55–66, 2011.
[77] A. E. Eiben, R. Hinterding, and Z. Michalewicz, Parameter control in evolutionary algorithms, *IEEE Trans. Evol. Comput.*, vol. 3, no. 2, pp. 124–141, 1999.
[78] R. Poláková, J. Tvrdík, and P. Bujok, Differential evolution with adaptive mechanism of population size according to current population diversity, *Swarm Evol. Comput.*, vol. 50, p. 100519, 2019.
[79] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, OpenAI Gym, arXiv preprint arXiv: 1606.01540, 2016.
[80] N. Hansen, A. Auger, S. Finck, and R. Ros, Real-parameter black-box optimization benchmarking 2010: Experimental setup, http://inria.hal.science/inria-00462481/, 2024.
[81] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, Stable-Baselines3: Reliable reinforcement learning implementations, *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.
[82] J. J. Liang, B. Y. Qu, P. N. Suganthan, and A. G. Hernández-Díaz, Problem definitions and evaluation criteria for the CEC 2013 special session on real-parameter optimization, Technical report, Computational Intelligence Laboratory, Zhengzhou University, China, and Nanyang Technological University, Singapore, 2012.

[83] R. Tanabe and A. Fukunaga, Evaluating the performance of SHADE on CEC 2013 benchmark problems, in *Proc. 2013 IEEE Congress on Evolutionary Computation*, Cancun, Mexico, 2013, pp. 1952–1959.
[84] J. Brest, S. Greiner, B. Boskovic, M. Mernik, and V. Zumer, Self-adapting control parameters in differential evolution: A comparative study on numerical benchmark problems, *IEEE Trans. Evol. Comput.*, vol. 10, no. 6, pp. 646–657, 2006.
[85] A. K. Qin and P. N. Suganthan, Self-adaptive differential evolution algorithm for numerical optimization, in *Proc. 2005 IEEE Congress on Evolutionary Computation*, Edinburgh, UK, 2005, pp. 1785–1791.
[86] L. Deng, C. Li, Y. Lan, G. Sun, and C. Shang, Differential evolution with dynamic combination based mutation operator and two-level parameter adaptation strategy, *Expert Syst. Appl.*, vol. 192, p. 116298, 2022.
[87] G. Sun, R. Han, L. Deng, C. Li, and G. Yang, Hierarchical structure-based joint operations algorithm for global optimization, *Swarm Evol. Comput.*, vol. 79, p. 101311, 2023.
[88] M. Abdel-Basset, R. Mohamed, M. Jameel, and M. Abouhawwash, Nutcracker optimizer: A novel nature-inspired metaheuristic algorithm for global optimization and engineering design problems, *Knowledge-Based Syst.*, vol. 262, p. 110248, 2023.
[89] Z. Meng and C. Yang, Two-stage differential evolution with novel parameter control, *Inf. Sci.*, vol. 596, pp. 321–342, 2022.
[90] J. Bader and E. Zitzler, HypE: An algorithm for fast hypervolume-based many-objective optimization, *Evol. Comput.*, vol. 19, no. 1, pp. 45–76, 2011.
[91] Z. Wang, C. Gehring, P. Kohli, and S. Jegelka, Batched large-scale Bayesian optimization in high-dimensional spaces, in *Proc. 21st Int. Conf. Artificial Intelligence and Statistics*, Lanzarote, Spain, 2018, pp. 745–754.



Wenjie Ding received the BEng degree from Hohai University, China in 2023. He is currently a master student at School of Mechanical and Vehicle Engineering, Chongqing University, China. His current research interests include reinforcement learning and its applications.



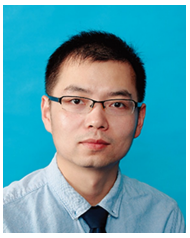
Mengtao Qian received the BEng degree from Chongqing University, China in 2021. He is currently a master student at School of Mechanical and Vehicle Engineering, Chongqing University, China. His current research interests include evolutionary algorithms and reinforcement learning.



Chao Lu received the PhD degree in industrial engineering from Huazhong University of Science and Technology, China in 2017. He is currently an associate professor of computer sciences at School of Computer Science, China University of Geosciences, China. His research interests include evolutionary algorithms and intelligent scheduling.



Jun Luo received the BEng and MEng degrees in mechanical engineering from Henan Polytechnic University, China in 1994 and 1997, respectively, and the PhD degree from Research Institute of Robotics, Shanghai Jiao Tong University, China in 2000. He is currently a professor at State Key Laboratory of Mechanical Transmission for Advanced Equipment, Chongqing University, China. His research areas include intelligent unmanned systems planning and control.



Jin Yi received the BEng and PhD degrees in industrial engineering from Huazhong University of Science and Technology (HUST), China in 2012 and 2017, respectively. He was a research fellow at Department of Industrial System Engineering and Management, National University of Singapore from 2017 to 2020. He is currently an associate professor at State Key Laboratory of Mechanical Transmission for Advanced Equipment, Chongqing University, China. His research interests include evolutionary algorithms and their applications in task assignment and planning for intelligent unmanned systems.



Huayan Pu received the MEng and PhD degrees in mechatronics engineering from Huazhong University of Science and Technology, China in 2007 and 2011, respectively. She is currently a professor at State Key Laboratory of Mechanical Transmission for Advanced Equipment, Chongqing University, China. Her current research interests include artificial intelligence, and intelligent unmanned systems planning and control.